https://youtu.be/W1cmUQQcVVk



ข้อเสนอโครงงาน

ระบบสั่งอาหารผ่านหน้าจอแท็บเล็ตด้วยตนเอง

Self-ordering System

จัดทำโดย

นางสาวธิญารัจน์ เตชธรรมเศรษฐ์ 67070077

นางสาวพัชณกัญญ์ คชาวุธากร 67070112

เสนอ

ผศ.ดร.มานพ พันธ์โคกกรวด

โครงงานนี้เป็นส่วนหนึ่งของรายวิชา 06066302 FUNDAMENTAL WEB PROGRAMMING

คณะเทคโนโลยีสารสนเทศ สาขาวิทยาการข้อมูลและการวิเคราะห์เชิงธุรกิจ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

ภาคเรียนที่ 1 ปีการศึกษา 2568

Table of Contents

ความเป็นมาของโครงงาน **6**

วัตถุประสงค์ **6**

ขอบเขตการทำงานของระบบ **7**

การทำงานแยกแต่ละ Actor **9**

3.1 ลูกค้า 9

3.2 บาริสต้า 9

3.3 แคชเชียร์ 9

USE case diagram **11**

ตั้งค่าการเริ่มต้นใช้งาน 11

เลือกหมวดหมู่อาหาร 12

ปรับแต่งรายการอาหาร ประเภทเครื่องดื่ม 13

เพิ่มรายการอาหารเข้าตะกร้าสินค้า 14

จัดการรายการอาหารในตะกร้า 15

ชำระเงิน 16

การจัดการออเดอร์ 17

แก้ไขจำนวนรายการอาหาร 18

รับชำระเงิน 19

ER Diagram **21**

Data Dictionary **23**

Category - ประเภทเมนู 24

Employees - พนักงาน 24

Ingredient - วัตถุดิบ 25

ItemOption - ตัวเลือกของเมนู 25

ItemOptionIngredient - วัตถุดิบที่ใช้ในตัวเลือก 26

Menu - เมนูอาหาร/เครื่องดื่ม 26

MenuIngredient - วัตถุดิบที่ใช้ในเมนู 27

Menu_OptionGroup - ความสัมพันธ์ระหว่างเมนูกับกลุ่มตัวเลือก 27

OptionGroup - กลุ่มของตัวเลือก 28

Orders - คำสั่งซื้อ 29

OrderItem - รายการเมนูในคำสั่งซื้อ 29

OrderItemOption - ตัวเลือกในเมนูที่ถูกสั่ง 30

OrderPromotion - โปรโมชันที่ใช้ในคำสั่งซื้อ 30

Payment - การชำระเงิน 30

Promotion - โปรโมชัน 31

เทคนิคและวิธีการใชนการสร้างองค์ประกอบ ***32***

ฟังก์ชั้นการทำงาน **32**

เพิ่มเข้าตะกร้า และ จัดการสินค้าในตะกร้า (พนักงานแคชเชียร์ และ ลูกค้า) 32

สำหรับจัดการคลังสินค้า (บาริสต้า) 34

การทำงานของระบบ 40

หน้าหลักของระบบ 40

การจัดการ modal ของเมนูต่างๆ 41

ลูกค้าและพนักงานแคชเชียร์ 44

หน้าเริ่มต้นการใช้งาน 44

การเลือกรับประทาน 44

หน้าหลักการใช้งานของลูกค้า (main) 45

หน้าตะกร้าสินค้า 49

หน้าตะกร้าสินค้า (การยืนยันสั่งซื้อสินค้า) 50

หน้าตะกร้าสินค้า (การยกเลิกสั่งซื้อสินค้า) 50

หน้าชำระเงิน (QR CODE) 51

หน้ายืนยันว่าชำระเงินเสร็จสิ้น 51

endpoint สำหรับส่งข้อมูลหรืออัปเดตข้อมูล (สำหรับลูกค้าและแคชเชียร์) 56

การอัปเดต ตะกร้าสินค้าเวลากดเพิ่มสินค้าเข้ามาในตะกร้า 56

การอัปเดต ตะกร้าสินค้าเวลากดเพิ่มสินค้าในตะกร้า 57

หน้าหลักของ bartender 58

ดูสินค้าในคลัง 60

ดู Stock สินค้าหมวดหมู่เบเกอรี่ 62

endpoint สำหรับส่งข้อมูลหรืออัปเดตข้อมูล (สำหรับบาริสต้า) 63

การอัปเดตสถานะคำสั่งซื้อ 63

จัดการสถานะเมนูเครื่องดื่ม 64

จัดการคลังสินค้าของเบเกอรี่ 64

คู่มือการใช้งาน 65

ลูกค้า 65

หน้าเริ่มต้นการใช้งาน 65

เลือกรับประทานที่ไหน 66

หน้าเลือกเมนู (main_for_customer) 67

หน้าเลือกเมนูสินค้า 68

หน้าตะกร้าสินค้า (cart) 69

หน้ายืนยันคำสั่งซื้อ 70

หน้าชำระเงิน 71

หน้ายืนยันการชำระเงิน หรือ ชำระเงินสำเร็จ 72

พนักงานแคชเชียร์ 73

หน้าเริ่มต้นของพนักงานแคชเชียร์ 73

หน้าหลักของพนักงานแคชเชียร์ (main_for_cashier) 74

หน้าเลือกเมนูและปรับแต่ง 75

หน้าตะกร้าสินค้า 76

หน้ายืนยันคำสั่งซื้อ 76

หน้าชำระเงิน 78

หน้ายืนยันการชำระเงิน หรือ ชำระเงินสำเร็จ 78

หน้าจัดการคลังสินค้าสำหรับพนักงานแคชเชียร์ 79

พนักงานบาริสต้า 83

หน้าหลักของพนักงานบาริสต้า.....83
หน้าจัดการคลังสินค้าของบาริสต้า 84

คู่มือการใช้งาน การติดตั้งระบบ 86

ปัญหาและแนวทางการปรับปรุง.....89

ความเป็นมาของโครงงาน

ในปัจจุบัน ธุรกิจร้านอาหารมีการแข่งขันที่รุนแรงมากขึ้น อันเป็นผลมาจากการเปลี่ยนแปลงของพฤติกรรมผู้บริโภคและการเติบโตของอุตสาหกรรมบริการด้านอาหาร ความคาดหวังของลูกค้าในเรื่องความรวดเร็ว ความถูกต้อง และคุณภาพของการบริการ กลายเป็นปัจจัยสำคัญที่ร้านอาหารจำเป็นต้องปรับตัวเพื่อให้สามารถตอบสนองต่อความต้องการดังกล่าวได้อย่างมีประสิทธิภาพ

วิธีการให้บริการในรูปแบบดั้งเดิม ซึ่งพนักงานทำหน้าที่รับออเดอร์ คิดเงิน และประสานงานกับครัว แม้จะสามารถรองรับการให้บริการได้ แต่ยังคงมีข้อจำกัดหลายประการ เช่น ความล่าช้าในช่วงเวลาที่มีลูกค้าจำนวนมาก ความผิดพลาดที่อาจเกิดขึ้นในการสื่อสารระหว่างลูกค้าและพนักงาน รวมถึงภาระค่าใช้จ่ายด้านแรงงานที่สูงขึ้น เพื่อแก้ไขปัญหาเหล่านี้ ร้านอาหารจำนวนมากเริ่มนำเทคโนโลยีดิจิทัลเข้ามาประยุกต์ใช้ โดยเฉพาะระบบสั่งอาหารผ่านหน้าจอแท็บเล็ตด้วยตนเอง (Self-ordering System) ซึ่งช่วยให้ลูกค้าสามารถเลือกเมนูอาหาร ปรับแต่งรายละเอียด ตรวจสอบราคา และยืนยันการสั่งซื้อได้ด้วยตนเอง โดยข้อมูลที่บันทึกจะถูกส่งไปยังครัวและระบบการชำระเงินโดยอัตโนมัติ ทำให้การดำเนินงานมีความถูกต้อง รวดเร็ว และลดภาระของพนักงาน

ด้วยเหตุนี้ โครงงานจึงมุ่งเน้นการพัฒนา Web Application สำหรับระบบสั่งอาหารผ่านหน้าจอแท็บเล็ตด้วยตนเองของร้านชิณกร คาเฟ่ โดยคำนึงถึงการออกแบบหน้าจอให้เหมาะสมต่อการใช้งานจริง ประสบการณ์ของผู้ใช้ (User Experience) และประสิทธิภาพในการทำงาน เพื่อช่วยสนับสนุนการดำเนินงานของร้านอาหาร ตอบสนองความต้องการของลูกค้า และสอดคล้องกับเป้าหมายเชิงธุรกิจในอนาคต

วัตถุประสงค์

โครงงานนี้มีวัตถุประสงค์เพื่อศึกษาและพัฒนา Web Application สำหรับระบบสั่งอาหารผ่านหน้าจอแท็บเล็ตด้วยตนเอง (Self-ordering System) เพื่อสนับสนุนการดำเนินงานของร้านกาแฟ “ชิณกร คาเฟ่” โดยมีวัตถุประสงค์หลัก ดังนี้

1. ออกแบบและพัฒนา Web Application สำหรับระบบสั่งอาหารผ่านหน้าจอแท็บเล็ตด้วยตนเอง (Self-ordering System) ของร้านชิณกร คาเฟ่ ที่ใช้งานง่าย โดยลูกค้าสามารถเรียกดูเมนู ปรับแต่งรายการ และยืนยันคำสั่งซื้อได้สะดวก รวดเร็ว และแม่นยำ ลดความผิดพลาดจากการสื่อสารระหว่างลูกค้าและพนักงาน
2. ช่วยลดภาระงานของพนักงานร้านกาแฟในการรับออเดอร์และคิดเงิน

3. เพิ่มทักษะและประสบการณ์ของผู้พัฒนาในการออกแบบและพัฒนา Web Application โดยประยุกต์ความรู้จากรายวิชา FUNDAMENTAL WEB PROGRAMMING
4. เปิดโอกาสให้เกิดการแลกเปลี่ยนความรู้ ทั้งภายในทีมและกับผู้มีประสบการณ์ในด้านเทคโนโลยีสารสนเทศ
5. พัฒนาทักษะวิชาการด้านการศึกษาค้นคว้า การเขียนรายงาน และการนำเสนอผลงาน พร้อมเสริมสร้างคุณลักษณะที่พึงประสงค์ เช่น ความรับผิดชอบ การทำงานร่วมกันเป็นทีม และความมั่นใจในตนเอง

ขอบเขตการทำงานของระบบ

Web Application สำหรับระบบสั่งอาหารผ่านหน้าจอแท็บเล็ตด้วยตนเอง (Self-ordering System) ของชิณกร คาเฟ่ ถูกออกแบบให้รองรับทั้งการทำงานฝั่งลูกค้าและฝั่งร้านอาหาร มุ่งเน้นให้ใช้งานง่าย สะดวก และแม่นยำ โดยขอบเขตการทำงานของระบบมี ดังนี้

1. ขั้นตอนการทำงานของระบบสั่งอาหารผ่านหน้าจอแท็บเล็ตด้วยตนเอง (Self-ordering System)
   1.1 การเลือกเมนูและปรับแต่งรายการอาหาร
   - ลูกค้าเข้าสู่ระบบผ่านหน้าจอสัมผัส (Kiosk) หรืออุปกรณ์อิเล็กทรอนิกส์
   - ลูกค้าเลือกระหว่างการสั่งสำหรับทานที่ร้าน หรือสั่งกลับบ้าน
   - ระบบจะแสดง E-Menu ที่มีหมวดหมู่อาหารชัดเจน พร้อมรูปภาพและรายละเอียดของเมนู
   - ลูกค้าสามารถเลือกเมนูที่ต้องการ ปรับเปลี่ยนตัวเลือก หรือเพิ่มหมายเหตุพิเศษสำหรับรายการนั้น ๆ ได้

   1.2 การยืนยันคำสั่งซื้อและการชำระเงิน
   - ลูกค้าตรวจสอบรายการอาหารที่สั่งและยอดรวมทั้งหมด
   - ระบบจะอำนวยความสะดวกในการชำระเงินผ่านช่องทางต่าง ๆ ที่รองรับ

   1.3 การส่งคำสั่งไปยังครัว
   - หลังจากชำระเงินแล้ว คำสั่งซื้อจะถูกส่งไปที่ห้องครัวโดยอัตโนมัติ

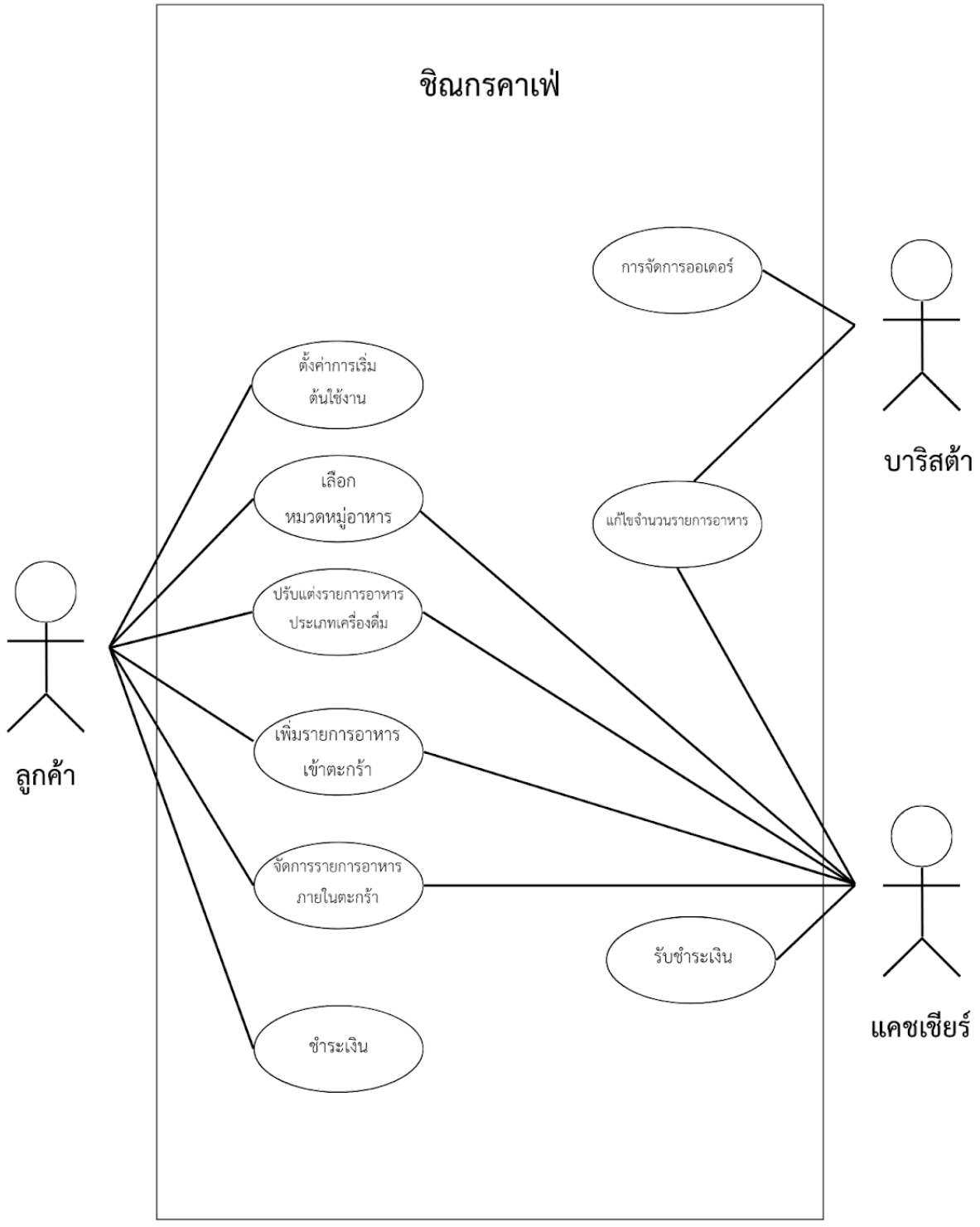
   1.4 การเตรียมและการรับสินค้า
   - บาริสต้าจะได้รับคำสั่งและเริ่มเตรียมอาหาร
   - เมื่ออาหารพร้อม แคชเชียร์จะประกาศคิวให้ลูกค้าทราบ เพื่อให้ลูกค้ามารับอาหารที่เคาน์เตอร์

   1.5 ระบบจัดการสต็อก
   - ทุกครั้งที่มีการสั่งซื้อเบเกอรี่ จำนวนสินค้าจะถูกหักออกจากสต็อกอัตโนมัติ

- บาริสต้าและแคชเชียร์สามารถแก้ไขจำนวนสินค้าเบเกอรี่ในระบบ และปิดรายการเมื่อสินค้าหมดได้
- หากเมนูเบเกอรี่หมด ลูกค้าจะไม่สามารถเลือกเมนูนั้นได้ ส่วนเมนูเครื่องดื่ม เมื่อวัตถุดิบหมด บาริสต้าหรือแคชเชียร์จะเป็นผู้ปิดรายการนั้นเอง

6

Use Case Diagram



การทำงานแยกแต่ละ Actor

3.1 ลูกค้า

3.1.1 ตั้งค่าเริ่มต้นใช้งาน

ลูกค้าสามารถสัมผัสหน้าจอบนแท็บเล็ตเพื่อเริ่มใช้งานระบบ โดยเลือกภาษาได้ทั้งไทยและอังกฤษ พร้อมทั้งเลือกสถานที่รับประทานอาหาร ได้แก่ รับประทานที่ร้าน หรือกลับไปรับประทานที่บ้าน

3.1.2 เลือกหมวดหมู่อาหาร

ลูกค้าสามารถเลือกหมวดหมู่อาหาร และรายการอาหารได้ โดยหมวดหมู่แยกเป็น เบเกอรี่ และเครื่องดื่ม เมื่อเลือกรายการอาหารจะแสดงรายละเอียดของอาหาร ราคา และจำนวนคงเหลือ

3.1.3 ปรับแต่งรายการอาหารประเภทเครื่องดื่ม

ลูกค้าสามารถปรับแต่งเมนูหมวดหมู่เครื่องดื่มได้ เช่น เพิ่มท็อปปิ้ง เลือกระดับความหวาน หรือใส่หมายเหตุพิเศษ

3.1.4 เพิ่มรายการอาหารเข้าตะกร้า

ลูกค้าสามารถเพิ่มรายการอาหารเข้าตะกร้าได้

3.1.5 จัดการรายการอาหารภายในตะกร้า

ลูกค้าสามารถแก้ไขจำนวน ลด ลบ เพิ่มรายการอาหาร รวมถึงยืนยันคำสั่งซื้อได้

3.1.6 ชำระเงิน

ลูกค้าสามารถเลือกวิธีการชำระเงิน และชำระเงินด้วยตนเองได้ผ่าน QR Code กรณีต้องการชำระเงินสด ต้องสั่งซื้อผ่านพนักงานแคชเชียร์เท่านั้น เมื่อการชำระเงินเสร็จสิ้น รายการอาหารที่ลูกค้าสั่งจะถูกส่งไปยังครัวโดยอัตโนมัติ

3.2 บาริสต้า

3.2.1 การจัดการออเดอร์

บาริสต้าสามารถตรวจสอบรายละเอียดออเดอร์ (เครื่องดื่ม/เบเกอรี่) และจัดเตรียมเมนูตามที่ลูกค้าต้องการ เมื่อทำเสร็จ บาริสต้าสามารถอัปเดตสถานะออเดอร์ว่า “เสร็จสิ้น” เพื่อดูออเดอร์ถัดไป

3.2.2 แก้ไขจำนวนรายการอาหารประเภทเบเกอรี่ (สามารถทำงานได้เหมือนกับแคชเชียร์)

บาริสต้าสามารถแก้ไขจำนวนเบเกอรี่ในระบบได้ โดยข้อมูลสินค้าเบเกอรี่คงเหลือจะแสดงตอนที่ลูกค้าเลือกซื้อสินค้าประเภทเบเกอรี่

3.3 แคชเชียร์

3.3.1 เลือกหมวดหมู่อาหาร (สามารถทำงานได้เหมือนกับลูกค้า)

ลูกค้าสามารถเลือกหมวดหมู่อาหาร และรายการอาหารได้ โดยหมวดหมู่แยกเป็น เบเกอรี่ และเครื่องดื่ม เมื่อเลือกรายการอาหารจะแสดงรายละเอียดของอาหาร ราคา และจำนวนคงเหลือ

3.3.2 ปรับแต่งรายการอาหารประเภทเครื่องดื่ม (สามารถทำงานได้เหมือนกับลูกค้า)

ลูกค้าสามารถปรับแต่งเมนูหมวดหมู่เครื่องดื่มได้ (เช่น เพิ่มท็อปปิ้ง เลือกระดับความหวาน หรือใส่หมายเหตุพิเศษ)

3.3.3 เพิ่มรายการอาหารเข้าตะกร้า (สามารถทำงานได้เหมือนกับลูกค้า)

ลูกค้าสามารถเพิ่มรายการอาหารเข้าตะกร้าได้

3.3.4 จัดการรายการอาหารภายในตะกร้า (สามารถทำงานได้เหมือนกับลูกค้า)

ลูกค้าสามารถแก้ไขจำนวน ลบรายการอาหาร หรือเพิ่มรายการอาหาร รวมถึงกดยืนยันคำสั่งซื้อได้

3.3.5 แก้ไขจำนวนรายการอาหารประเภทเบเกอรี่ (สามารถทำงานได้เหมือนกับบาริสต้า)

แคชเชียร์สามารถแก้ไขจำนวนเบเกอรี่ในระบบได้ โดยข้อมูลสินค้าเบเกอรี่คงเหลือจะแสดงเมื่อลูกค้าเลือกซื้อสินค้าประเภทเบเกอรี่

3.3.6 รับชำระเงิน

แคชเชียร์รับชำระเงินจากลูกค้า โดยลูกค้าสามารถเลือกวิธีการชำระเงินได้ 2 ประเภท คือ ชำระเงินสด และ ชำระเงินผ่าน QR Code

USE case diagram

<table>
<tr><td>Use case Name:</td><td>ตั้งค่าการเริ่มต้นใช้งาน</td><td>ID: 01</td></tr>
<tr><td>Scenario:</td><td colspan="2">ผู้ใช้เริ่มต้นใช้งานระบบสั่งอาหารเป็นครั้งแรก โดยต้องทำการสัมผัสหน้าจอ
ะ เลือกประเภทการให้บริการ (รับประทานที่ร้าน / ซื้อกลับบ้าน)</td></tr>
<tr><td>Triggering Event:</td><td colspan="2">เมื่อผู้ใช้ต้องการ "เริ่มต้นสั่งอาหาร" หรือเข้าใช้งานระบบเป็นครั้งแรก</td></tr>
<tr><td>Brief Description:</td><td colspan="2">เมื่อผู้ใช้กด “เริ่มต้นสั่งอาหาร” ระบบจะแสดงตัวเลือกให้ผู้ใช้
เลือกประเภทการให้บริการ (รับประทานที่ร้าน / ซื้อกลับบ้าน)
จากนั้นระบบจะบันทึกข้อมูลไว้</td></tr>
<tr><td>Actor(s):</td><td colspan="2">ลูกค้า</td></tr>
<tr><td>Related Use Cases:</td><td colspan="2">1. เริ่มต้นใช้งานระบบ
2. บันทึกการตั้งค่าเริ่มต้น</td></tr>
<tr><td>Stakeholders:</td><td colspan="2">ลูกค้า</td></tr>
<tr><td>Pre - Conditions:</td><td colspan="2">1. ระบบพร้อมทำงานและโหลดประเภทการให้บริการที่รองรับ
2. หน้าเลือกรูปแบบการให้บริการปรากฏขึ้นเมื่อเริ่มใช้งานครั้งแรก</td></tr>
<tr><td>Post - Conditions:</td><td colspan="2">1. ประเภทการให้บริการที่เลือกจะถูกบันทึกเป็นค่าเริ่มต้นในคำสั่งซื้อ</td></tr>
<tr><td rowspan="2">Flow of Events:</td><td>Actor</td><td>System</td></tr>
<tr><td>1. เลือก “เริ่มต้นใช้งาน”
2. เลือกประเภทการให้บริการ (รับประทานที่ร้าน/ซื้อกลับบ้าน)</td><td>1. ระบบแสดงเมนูแนะนำ
2. แสดงตัวเลือกประเภทการให้บริการ (รับประทานที่ร้าน / ซื้อกลับบ้าน)
3. บันทึกประเภทการให้บริการที่ลูกค้าเลือกลงในคำสั่งซื้อ</td></tr>
<tr><td>Exception Conditions:</td><td colspan="2">1. ไม่สามารถโหลดรายการภาษาได้
2. เกิดข้อผิดพลาดในการโหลดไฟล์ภาษา</td></tr>
</table>

Use case Name:	**เลือกหมวดหมู่อาหาร**	**ID:** 02
Scenario:	ผู้ใช้ดูรายการเมนูอาหาร และรายละเอียดของรายการอาหาร	
Triggering Event:	เมื่อผู้ใช้ต้องการดูรายการอาหาร และรายละเอียดของรายการอาหาร	
Brief Description:	เมื่อผู้ใช้เข้าสู่หน้าจอเลือกอาหาร ระบบจะแสดงประเภทหมวดหมู่อาหาร เมื่อผู้ใช้เลือกหมวดหมู่ที่ต้องการ (เช่น เมนูแนะนำ, เมนูพิเศษตามฤดูกาล, ชา, ชานม, กาแฟ, กาแฟ signal หรือเบเกอรี่) ระบบจะแสดงรายการเมนูย่อยที่เกี่ยวข้อง และเมื่อผู้ใช้เลือกเมนูใดเมนูหนึ่ง ระบบจะแสดงรายละเอียดของเมนูนั้น	
Actor(s):	ลูกค้า	
Related Use Cases:	**ID:** 01 ตั้งค่าการเริ่มต้นใช้งาน **ID:** 03 ปรับแต่งรายการอาหารประเภทเครื่องดื่ม **ID:** 04 เพิ่มรายการอาหารเข้าตะกร้า	
Stakeholders:	ลูกค้า	
Pre - Conditions:	1. ผู้ใช้ผ่านขั้นตอนตั้งค่าเริ่มต้นใช้งานเรียบร้อยแล้ว 2. ระบบมีการเชื่อมต่อกับฐานข้อมูลเมนูอาหาร	
Post - Conditions:	1. ระบบจะแสดงผลตามที่ผู้ใช้เลือก (แสดงหมวดหมู่, รายการอาหาร หรือรายละเอียดของอาหาร)	
Flow of Events:	**Actor**	**System**
	1. เลือกหมวดหมู่อาหารที่ต้องการ 2. เลือกรายการอาหารที่ต้องการ	1. แสดงหมวดหมู่อาหาร เช่น เมนูแนะนำ, เมนูพิเศษตามฤดูกาล, ชา, ชานม, กาแฟ, กาแฟ signal หรือเบเกอรี่ 2. โหลดข้อมูลรายการอาหาร 3. ระบบแสดงรายการอาหารย่อยในหมวดหมู่ที่ผู้ใช้ต้องการ 4. โหลดข้อมูลรายละเอียดของอาหารจากฐานข้อมูล 5. ระบบแสดงรายละเอียดของอาหารที่ผู้ใช้ต้องการ

Exception Conditions:	1. วัตถุดิบไม่เพียงพอระบบแจ้ง “วัตถุดิบหมด ไม่สามารถเพิ่มเข้าตะกร้าได้” 2. ระบบเกิดการขัดข้อง การเชื่อมต่อเครือข่ายเกิดปัญหา

Use case Name:	**ปรับแต่งรายการอาหาร ประเภทเครื่องดื่ม**	**ID:** 03
Scenario:	ผู้ใช้เลือกหมวดหมู่เครื่องดื่ม (เช่น ชา, ชานม, กาแฟ, กาแฟ signal) จากหน้าจอ Tablet โดยผู้ใช้จะสามารถปรับแต่งเครื่องดื่มได้ตามความต้องการ เช่น ระดับความหวาน, เพิ่มนมโอ๊ต หรือกาแฟช็อต เมื่อเสร็จสิ้นแล้วกด “เพิ่มเข้าตะกร้า”	
Triggering Event:	เมื่อผู้ใช้เลือกรายการอาหารประเภทเครื่องดื่ม	
Brief Description:	ระบบจะตรวจสอบสต็อกวัตถุดิบ หากมีเพียงพอ ผู้ใช้จะสามารถเลือกและเพิ่มเมนูเครื่องดื่ม (ที่ได้รับการปรับแต่งแล้ว) ลงในตะกร้าได้ หากมีวัตถุดิบไม่เพียงพอ ระบบจะแสดงสีเทาที่ตัวเลือกนั้น และผู้ใช้จะไม่สามารถกดเลือกได้	
Actor(s):	ลูกค้า	
Related Use Cases:	**ID:** 02 เลือกหมวดหมู่อาหาร **ID:** 08 ชำระเงิน	
Stakeholders:	ลูกค้า, บาริสต้า	
Pre - Conditions:	1. ผู้ใช้เลือกหมวดหมู่เครื่องดื่ม (เช่น ชา, ชานม, กาแฟ, กาแฟ signal) จากหน้าจอ Tablet	
Post - Conditions:	1. ระบบบันทึกรายการเครื่องดื่มลงในตะกร้าสินค้า	
Flow of Events:	**Actor**	**System**
	1. เลือกรายการเครื่องดื่มที่ต้องการ 2. ปรับแต่งเครื่องดื่ม	1. โหลดข้อมูลรายละเอียดของเครื่องดื่มจากฐานข้อมูล 2. แสดงรายละเอียดเครื่องดื่ม

	(เช่น ความหวาน, ร้อน, เย็น, ฯลฯ) 3. ระบุจำนวนแก้วที่ต้องการ 4. กด “เพิ่มเข้าตะกร้า”	(เช่น ชื่อ, รูปภาพ, ราคา, ตัวเลือกการปรับแต่ง) 3. ตรวจสอบการปรับแต่ง และคำนวณราคาที่ปรับตามการเลือก 4. เพิ่มรายการเครื่องดื่มเข้าตะกร้าสินค้า
Exception Conditions:	1. หากวัตถุดิบไม่เพียงพอ ผู้ใช้จะไม่สามารถเลือกปรับแต่งหรือเพิ่มเข้าตะกร้าสินค้าได้ 2. ระบบเกิดการขัดข้อง การเชื่อมต่อเครือข่ายเกิดปัญหา	

Use case Name:	**เพิ่มรายการอาหารเข้าตะกร้าสินค้า**	**ID:** 04
Scenario:	ผู้ใช้เลือกรายการอาหารใด ๆ และกด “เพิ่มเข้าตะกร้า”	
Triggering Event:	เมื่อผู้ใช้กด “เพิ่มเข้าตะกร้า”	
Brief Description:	ระบบบันทึกข้อมูลรายการอาหารลงในตะกร้าสินค้า หาก “สินค้าหมด” จะไม่สามารถเลือกสินค้าได้ตั้งแต่แรก	
Actor(s):	ลูกค้า	
Related Use Cases:	**ID:** 02 เลือกหมวดหมู่อาหาร **ID:** 08 ชำระเงิน	
Stakeholders:	ลูกค้า	
Pre - Conditions:	1. ผู้ใช้เลือกสินค้าใด ๆ และกด “เพิ่มเข้าตะกร้า”	
Post - Conditions:	1. ระบบบันทึกข้อมูลรายการอาหารลงในตะกร้าสินค้า	
	Actor	**System**

Flow of Events:	1. เลือกรายการอาหารที่ต้องการ 2. กด “เพิ่มลงตะกร้า”	1. โหลดข้อมูลรายละเอียดของอาหารจากฐานข้อมูล 2. แสดงรายละเอียดรายการอาหาร (เช่น ชื่อ, รูปภาพ, ราคา, ตัวเลือกการปรับแต่ง) 3. บันทึกรายการอาหารลงในตะกร้าสินค้า
Exception Conditions:	1. หากวัตถุดิบไม่เพียงพอ ผู้ใช้จะไม่สามารถเลือกปรับแต่งหรือเพิ่มเข้าตะกร้าสินค้าได้ 2. ระบบเกิดการขัดข้อง การเชื่อมต่อเครือข่ายเกิดปัญหา	

Use case Name:	**จัดการรายการอาหารในตะกร้า**	**ID:** 05
Scenario:	ผู้ใช้ดูรายการสินค้าภายในตะกร้า หรือผู้ใช้ต้องการแก้ไขจำนวนสินค้าภายในตะกร้าสินค้า	
Triggering Event:	เมื่อผู้ใช้กด “ตะกร้าสินค้า”	
Brief Description:	ในตะกร้าสินค้า ผู้ใช้สามารถดูรายการอาหารที่เลือก และสามารถปรับจำนวนสินค้าในตะกร้าได้ (เช่น เพิ่มรายการสินค้า, ลดรายการสินค้า, ลบรายการสินค้า) และสามารถเลือกได้ว่าจะยืนยันการสั่งซื้อ หรือถอยหลังเพื่อเลือกรายการอาหารต่อ	
Actor(s):	ลูกค้า	
Related Use Cases:	**ID:** 03 ปรับแต่งรายการอาหาร ประเภทเครื่องดื่ม **ID:** 04 เพิ่มรายการอาหารเข้าตะกร้าสินค้า **ID:** 06 ชำระเงิน **ID:** 09 รับชำระเงิน	
Stakeholders:	ลูกค้า	
Pre - Conditions:	1. ผู้ใช้กด “ตะกร้าสินค้า”	

Post - Conditions:	1. ระบบบันทึกรายการอาหารที่ปรับลงในตะกร้าสินค้า	
Flow of Events:	**Actor**	**System**
	1. กดปุ่ม “ตะกร้า” 2. แก้ไขรายการอาหารที่ต้องการ	1. โหลดข้อมูลตะกร้าปัจจุบันจากฐานข้อมูล 2. แสดงข้อมูลตะกร้าปัจจุบัน 3. บันทึกการเปลี่ยนแปลงของตะกร้า
Exception Conditions:	1. หากวัตถุดิบไม่เพียงพอ ผู้ใช้จะไม่สามารถเลือกปรับแต่งหรือเพิ่มเข้าตะกร้าสินค้าได้ 2. ระบบเกิดการขัดข้อง การเชื่อมต่อเครือข่ายเกิดปัญหา	

Use case Name:	**ชำระเงิน**	**ID:** 06
Scenario:	ผู้ใช้ชำระเงิน	
Triggering Event:	เมื่อผู้ใช้กดปุ่ม “ยืนยันคำสั่งซื้อ”	
Brief Description:	เมื่อผู้ใช้ยืนยันการรับสินค้า ระบบจะแสดงยอดรวมและวิธีการชำระเงิน (ของลูกค้าชำระได้เพียง Qr-พร้อมเพย์) เมื่อผู้ใช้เลือกวิธีระแล้ว ระบบจะประมวลผลการจ่ายเงิน หากสำเร็จจะบันทึกออเดอร์ลงบนฐานข้อมูล และไม่สำเร็จจะย้อนกลับไปหน้าเริ่มต้น	
Actor(s):	ลูกค้า	
Related Use Cases:	**ID:** 05 จัดการรายการอาหารในตะกร้า **ID:** 07 การจัดการออเดอร์	
Stakeholders:	ลูกค้า, บาเทนเดอร์	

Pre - Conditions:	1. ระบบเชื่อมต่อกับ Payment Gateway	
Post - Conditions:	1. บันทึกข้อมูลออเดอร์ลงบนฐานข้อมูล	
Flow of Events:	**Actor**	**System**
	1. กดปุ่ม “ยืนยันคำสั่งซื้อ” 2. เลือกช่องทางการชำระเงิน 3. ดำเนินการชำระเงิน 4. รับข้อความแจ้งผลการทำรายการจากระบบ	1. ดึงข้อมูลคำสั่งซื้อจากฐานข้อมูล 2. แสดงยอดรวมสินค้า 3. สร้างฟอร์มสำหรับการชำระเงิน 4. แสดงฟอร์มสำหรับการชำระเงิน (Qr-code) 5. ตรวจสอบความถูกต้องของข้อมูลการชำระเงิน 6. บันทึกข้อมูลออเดอร์ในฐานข้อมูล 7. บันทึกข้อมูลชำระเงินในฐานข้อมูล
Exception Conditions:	1. ระบบเกิดการขัดข้อง การเชื่อมต่อเครือข่ายเกิดปัญหา 2. ไม่สามารถบันทึกคำสั่งซื้อในฐานข้อมูล	

Use case Name:	**การจัดการออเดอร์**	ID: 07
Scenario:	บาริสต้าดูออเดอร์ดูรายการอาหารที่ลูกค้าสั่ง	
Triggering Event:	บาริสต้าต้องการดูรายการอาหารที่ลูกค้าสั่ง	
Brief Description:	เมื่อคำสั่งซื้อถูกชำระเงินจากฝั่งลูกค้า ระบบจะส่งข้อมูลไปยัง KDS ที่เคาน์เตอร์กาแฟ บาริสต้าจะตรวจสอบออเดอร์และเริ่มทำทันที เมื่อทำเสร็จจะกด “เสร็จสิ้น” ระบบจะเปลี่ยนสถานะของออเดอร์ เมื่อมีออเดอร์ใหม่ ระบบจะแสดงอัตโนมัติ	
Actor(s):	บาริสต้า	
Related Use Cases:	**ID:** 06 ชำระเงิน **ID:** 08 แก้ไขจำนวนสินค้า **ID:** 09 รับชำระเงิน	

Stakeholders:	บาริสต้า, แคชเชียร์	
Pre - Conditions:	1. ออเดอร์ถูกสร้างและสถานะเป็น “กำลังดำเนินการ” 2. ระบบ KDS พร้อมใช้งานและเชื่อมต่อกับฐานข้อมูลคำสั่งซื้อ	
Post - Conditions:	1. สถานะออเดอร์ถูกเปลี่ยนเป็น “เสร็จสิ้น”	
Flow of Events:	**Actor**	**System**
	1. ตรวจสอบรายการอาหารใหม่บน KDS 2. กด “เสร็จสิ้น”	1. โหลดข้อมูลออเดอร์สินค้าใหม่ จากฐานข้อมูล 2. แสดงรายการสินค้าภายในแต่ละออเดอร์ 3. อัปเดตสถานะออเดอร์เป็น “เสร็จสิ้น” ในฐานข้อมูล 4. ลบออเดอร์ที่เสร็จสิ้นออกจาก หน้าจอ
Exception Conditions:	1. ระบบเกิดการขัดข้อง การเชื่อมต่อเครือข่ายเกิดปัญหา 2. ไม่สามารถดึงข้อมูลคำสั่งซื้อในฐานข้อมูลได้	

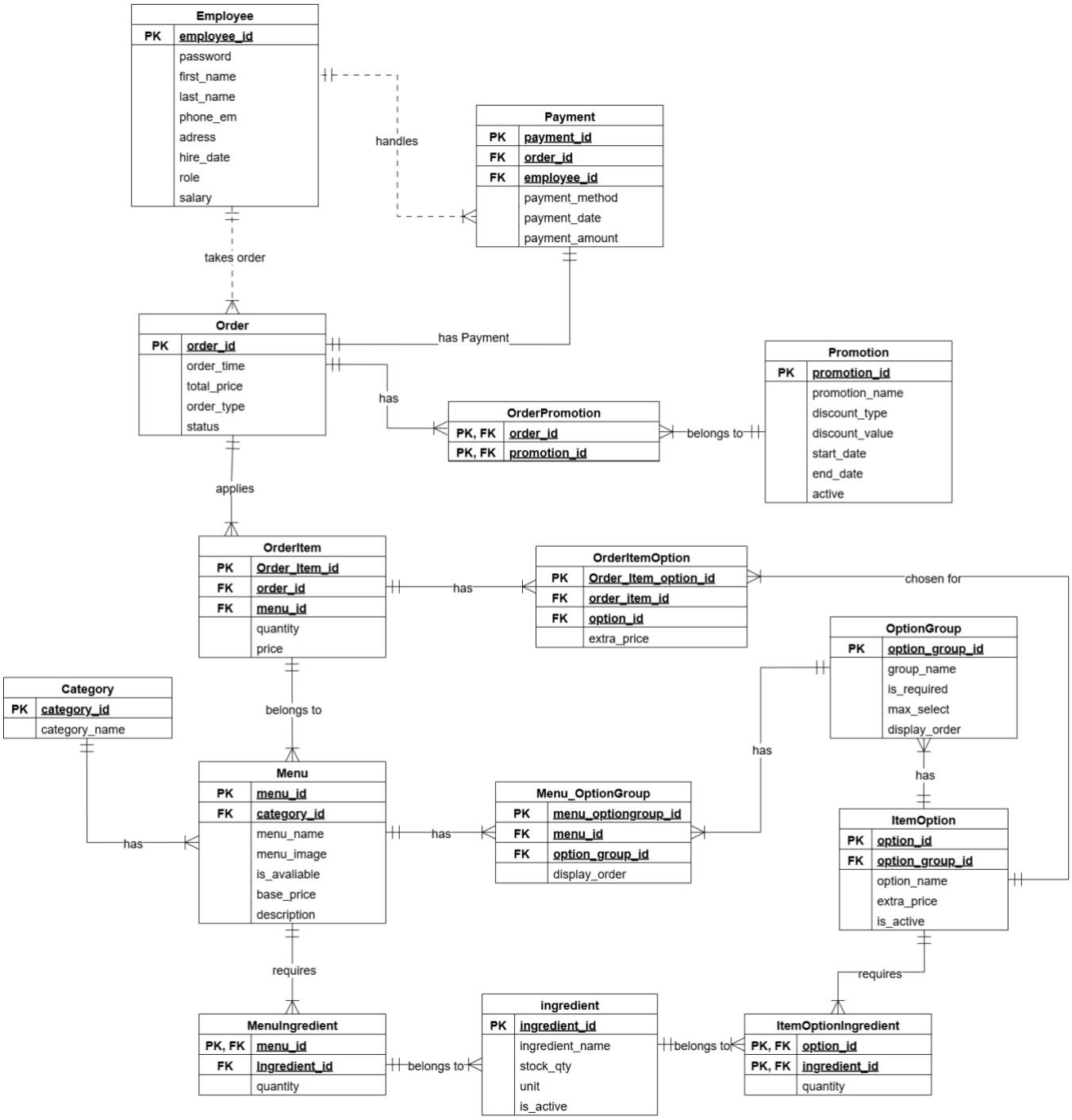
Use case Name:	**แก้ไขจำนวนรายการอาหาร**	ID: 08
Scenario:	บาริสต้าหรือแคชเชียร์อัปเดตจำนวนสินค้า	
Triggering Event:	บาริสต้าหรือแคชเชียร์ต้องการอัปเดตจำนวนสินค้า	
Brief Description:	บาริสต้าหรือแคชเชียร์สามารถแก้ไขจำนวนสินค้าในระบบ ซึ่งระบบจะอัปเดตจำนวนสินค้าในฐานข้อมูลทันที และแสดงจำนวนสินค้าที่ปรับใหม่ให้ลูกค้าเห็นบนหน้าจอ ในกรณีที่ประเภทหารเป็น “เบเกอรี่” เมื่อมีการมีคำสั่งซื้อเสร็จสิ้น จำนวนขนมจะลดลงโดยอัตโนมัติ	

<table>
<tr><td>Actor(s):</td><td colspan="2">บาริสต้า, แคชเชียร์</td></tr>
<tr><td>Related Use Cases:</td><td colspan="2">ID: 04 เพิ่มรายการเข้าตะกร้าสินค้า
ID: 06 ชำระเงิน
ID: 07 การจัดการออเดอร์
ID: 09 รับชำระเงิน</td></tr>
<tr><td>Stakeholders:</td><td colspan="2">บาริสต้า, แคชเชียร์</td></tr>
<tr><td>Pre - Conditions:</td><td colspan="2">1. รายการอาหารสามารถแก้ไขสถานะเมนูได้
2. รายการอาหารสามารถเพิ่ม / ลด จำนวนได้</td></tr>
<tr><td>Post - Conditions:</td><td colspan="2">1. จำนวนสินค้าในฐานข้อมูลเพิ่มขึ้นหรือลดลง
2. ในกรณีรายการอาหารที่สามารถปิดเมนูได้ สถานะเมนูจะต้องสอดคล้องกับ าานะ
เมนูในฐานข้อมูล</td></tr>
<tr><td rowspan="2">Flow of Events:</td><td>Actor</td><td>System</td></tr>
<tr><td>1. กดปุ่ม “คลังสินค้า”
2. เลือกหมวดหมู่รายการอาหาร
ที่ต้องการแก้ไข
3. แก้ไขจำนวนสินค้าหรืออัปเดต
สถานะเมนู (เปิด / ปิด)</td><td>1. โหลดข้อมูลสินค้าคงเหลือตามหมวดหมู่ (เครื่องดื่มหรือเบเกอรี่)
2. บันทึกจำนวนสินค้าในฐานข้อมูล
3. บันทึกสถานะเมนูในฐานข้อมูล</td></tr>
<tr><td>Exception Conditions:</td><td colspan="2">1. ระบบเกิดการขัดข้อง การเชื่อมต่อเครือข่ายเกิดปัญหา
2. เมื่อข้อมูลจำนวนสินค้า เป็นค่าลบ</td></tr>
</table>

Use case Name:	**รับชำระเงิน**	**ID:** 09
Scenario:	พนักงานรับชำระเงิน	
Triggering Event:	เมื่อพนักงานต้องการสร้างการ “ชำระเงิน” ให้ลูกค้า	

<table>
<tr><td>Brief Description:</td><td colspan="2">เมื่อลูกค้ายืนยันการรับสินค้า ระบบจะแสดงยอดรวมและตัวเลือกวิธีชำระเงินให้พนักงานเลือกแทนลูกค้า เมื่อพนักงานเลือกวิธีชำระแล้ว ระบบจะประมวลผลการจ่ายเงิน หากสำเร็จจะออกใบเสร็จและอัปเดตสถานะคำสั่งซื้อเป็น “ชำระแล้ว”</td></tr>
<tr><td>Actor(s):</td><td colspan="2">แคชเชียร์</td></tr>
<tr><td>Related Use Cases:</td><td colspan="2">ID: 05 จัดการรายการอาหารในตะกร้า
ID: 07 การจัดการออเดอร์</td></tr>
<tr><td>Stakeholders:</td><td colspan="2">แคชเชียร์, ลูกค้า</td></tr>
<tr><td>Pre - Conditions:</td><td colspan="2">1. ระบบเชื่อมต่อกับ Payment Gateway</td></tr>
<tr><td>Post - Conditions:</td><td colspan="2">1. บันทึกข้อมูลออเดอร์ลงบนฐานข้อมูล</td></tr>
<tr><td rowspan="2">Flow of Events:</td><td>Actor</td><td>System</td></tr>
<tr><td>1. กดปุ่ม “ยืนยันคำสั่งซื้อ”
2. เลือกช่องทางการชำระเงิน (Qr-code / เงินสด)
3. ดำเนินการชำระเงิน
4. รับข้อความแจ้งผลการทำรายการจากระบบ</td><td>1. ดึงข้อมูลคำสั่งซื้อจากฐานข้อมูล
2. แสดงยอดรวมสินค้า
3. สร้างฟอร์มสำหรับการชำระเงิน
4. แสดงฟอร์มสำหรับการชำระเงิน (Qr-code)
5. ตรวจสอบความถูกต้องของข้อมูลการชำระเงิน
6. บันทึกข้อมูลออเดอร์ในฐานข้อมูล
7. บันทึกข้อมูลชำระเงินในฐานข้อมูล</td></tr>
<tr><td>Exception Conditions:</td><td colspan="2">1. ระบบเกิดการขัดข้อง การเชื่อมต่อเครือข่ายเกิดปัญหา
2. ไม่สามารถดึงข้อมูลคำสั่งซื้อในฐานข้อมูลได้</td></tr>
</table>

ER Diagram


Employee
PK employee_id
password
first_name
last_name
phone_em
adress
hire_date
role
salary
handles
Payment
PK payment_id
FK order_id
FK employee_id
payment_method
payment_date
payment_amount
takes order
Order
PK order_id
order_time
total_price
order_type
status
has Payment
has
Promotion
PK promotion_id
promotion_name
discount_type
discount_value
start_date
end_date
active
OrderPromotion
PK, FK order_id
PK, FK promotion_id
belongs to
applies
OrderItem
PK Order_Item_id
FK order_id
FK menu_id
quantity
price
has
OrderItemOption
PK Order_Item_option_id
FK order_item_id
FK option_id
extra_price
chosen for
OptionGroup
PK option_group_id
group_name
is_required
max_select
display_order
Category
PK category_id
category_name
belongs to
has
has
Menu
PK menu_id
FK category_id
menu_name
menu_image
is_avaliable
base_price
description
has
Menu_OptionGroup
PK menu_optiongroup_id
FK menu_id
FK option_group_id
display_order
has
ItemOption
PK option_id
FK option_group_id
option_name
extra_price
is_active
requires
requires
MenuIngredient
PK, FK menu_id
FK Ingredient_id
quantity
belongs to
ingredient
PK ingredient_id
ingredient_name
stock_qty
unit
is_active
belongs to
ItemOptionIngredient
PK, FK option_id
PK, FK ingredient_id
quantity

Data Dictionary

Table name	Description
Category	*ประเภทเมนู*
Employees	*พนักงาน*
Ingredient	*วัตถุดิบ*
ItemOption	*ตัวเลือกของเมนู*
ItemOptionIngredient	*วัตถุดิบที่ใช้ในตัวเลือก*
Menu	*เมนูอาหาร/เครื่องดื่ม*
MenuIngredient	*วัตถุดิบที่ใช้ในเมนู*
Menu_OptionGroup	*ความสัมพันธ์ระหว่างเมนูกับกลุ่มตัวเลือก*
OptionGroup	*กลุ่มของตัวเลือ*
Orders	*คำสั่งซื้อ*
OrderItem	*รายการเมนูในคำสั่งซื้อ*
OrderItemOption	*ตัวเลือกในเมนูที่ถูกสั่ง*
OrderPromotion	*โปรโมชันที่ใช้ในคำสั่งซื้อ*
Payment	*การชำระเงิน*
Promotion	*โปรโมชัน*

Category - ประเภทเมนู			
Attribute	**Description**	**Data Type**	**Constraint**
category_id	รหัสประเภทเมนู	INT	Primary Key
category_name	ชื่อประเภทเมนู เช่น “กาแฟ”, “ชา”, “เบเกอรี่”	VACHAR(50)	

Employees - พนักงาน			
Attribute	**Description**	**Data Type**	**Constraint**
employee_id	รหัสพนักงาน	TEXT	(Primary Key)
password	รหัสผ่านสำหรับเข้าสู่ระบบ	VARCHAR(50)	
first_name	ชื่อจริงพนักงาน	VARCHAR(50)	
last_name	นามสกุลพนักงาน	VARCHAR(50)	
phone_em	เบอร์โทรศัพท์	VARCHAR(10)	
address	ที่อยู่พนักงาน	VARCHAR(100)	
hire_date	วันที่เริ่มงาน	DATETIME	
role	ตำแหน่งงาน เช่น “แคชเชียร์”, “บาริสต้า”, “ผู้จัดการ”	VARCHAR(50)	
salary	เงินเดือนพนักงาน	INT	

Ingredient - วัตถุดิบ			
Attribute	**Description**	**Data Type**	**Constraint**
ingredient_id	รหัสวัตถุดิบ	INT	Primary Key
ingredient_name	ชื่อวัตถุดิบ เช่น “นม”, “กาแฟ”, “น้ำตาล”	VARCHAR(100)	
stock_qty	ปริมาณคงเหลือในสต็อก	DECIMAL(10,2)	
unit	หน่วยนับ	VARCHAR(10)	
is_active	สถานะการใช้งาน(1=มีของ, 0=ไม่มีของ)	INT	

ItemOption - ตัวเลือกของเมนู			
Attribute	**Description**	**Data Type**	**Constraint**
option_id	รหัสตัวเลือก	INT	Primary Key
option_group_id	รหัสกลุ่มตัวเลือก	INT	Foreign Key
option_name	ชื่อตัวเลือก เช่น “เพิ่มช็อต”, “ไม่ใส่น้ำตาล”	VARCHAR(100)	
extra_price	ราคาที่เพิ่มขึ้นจากการเลือกตัวเลือกนี้	DECIMAL(10,2)	

is_active	สถานะการใช้งาน(1=มีของ, 0=ไม่มีของ)	INT	

ItemOptionIngredient - วัตถุดิบที่ใช้ในตัวเลือก			
Attribute	**Description**	**Data Type**	**Constraint**
option_id	รหัสตัวเลือก	INT	Foreign Key
ingredient_id	รหัสวัตถุดิบ	INT	Foreign Key
quantity	ปริมาณวัตถุดิบที่ใช้ในตัวเลือกนี้	DECIMAL(10,2)	
unit	หน่วยนับของวัตถุดิบ	VARCHAR(20)	

Menu - เมนูอาหาร/เครื่องดื่ม			
Attribute	**Description**	**Data Type**	**Constraint**
menu_id	รหัสเมนู	INT	Primary Key
category_id	รหัสประเภทเมนู	INT	Foreign Key
menu_name	ชื่อเมนู เช่น “ลาเต้เย็น”, “ชาเขียวปั่น”	VARCHAR(50)	
menu_image	รูปภาพ	VARCHAR(50)	

is_avaliable	สถานะพร้อมขาย(1=ขายได้, 0=หมด)	INT	
base_price	ราคาพื้นฐานของเมนู	INT	
description	รายละเอียดเพิ่มเติมของเมนู	TEXT	

MenuIngredient - วัตถุดิบที่ใช้ในเมนู			
Attribute	**Description**	**Data Type**	**Constraint**
menu_id	รหัสเมนู	INT	Foreign Key
ingredient_id	รหัสวัตถุดิบ	INT	Foreign Key
quantity	ปริมาณที่ใช้ต่อหนึ่งหน่วยเมนู	DECIMAL(10,2)	
unit	หน่วยนับของวัตถุดิบ	VARCHAR(20)	

Menu_OptionGroup - ความสัมพันธ์ระหว่างเมนูกับกลุ่มตัวเลือก			
Attribute	**Description**	**Data Type**	**Constraint**
menu_optiongroup_id	รหัสความสัมพันธ์	INT	Primary Key
menu_id	รหัสเมนู	INT	Foreign Key

option_group_id	รหัสกลุ่มตัวเลือก	INT	Foreign Key
is_required	ระบุว่าจำเป็นต้องเลือกหรือไม่	INT	
display_order	ลำดับการแสดงผลของกลุ่มตัวเลือก	INT	

OptionGroup - กลุ่มของตัวเลือก			
Attribute	**Description**	**Data Type**	**Constraint**
option_group_id	รหัสกลุ่มตัวเลือก	INT	Primary Key
group_name	ชื่อกลุ่ม เช่น “ขนาดแก้ว”, “ระดับความหวาน”	VARCHAR(100)	
is_required	ต้องเลือกหรือไม่	INT	
max_select	จำนวนตัวเลือกสูงสุดที่เลือกได้	INT	
display_order	ลำดับการแสดงในหน้าเมนู	INT	

Orders - คำสั่งซื้อ			
Attribute	**Description**	**Data Type**	**Constraint**
order_id	รหัสคำสั่งซื้อ	TEXT	Primary Key
order_time	วันที่และเวลาที่สั่ง	DATETIME	
total_price	ราคารวมทั้งหมด	INT	
order_type	ประเภทคำสั่งซื้อ เช่น “สั่งหน้าร้าน”, “สั่งกลับบ้าน”	TEXT	

OrderItem - รายการเมนูในคำสั่งซื้อ			
Attribute	**Description**	**Data Type**	**Constraint**
order_item_id	รหัสรายการ	INT	Primary Key
order_id	รหัสคำสั่งซื้อ	TEXT	Foreign Key
menu_id	รหัสเมนูที่สั่ง	INT	
quantity	จำนวนที่สั่ง	INT	
price	ราคารวม ต่อ เมนูนั้น	INT	

OrderItemOption - ตัวเลือกในเมนูที่ถูกสั่ง			
Attribute	**Description**	**Data Type**	**Constraint**
order_item_option_id	รหัสตัวเลือกในออเดอร์	INT	Primary Key
order_item_id	รหัสรายการเมนู	INT	Foreign Key
option_id	รหัสตัวเลือก	INT	Foreign Key
extra_price	ราคาที่เพิ่มขึ้นจากตัวเลือกนี้	INT	

OrderPromotion - โปรโมชันที่ใช้ในคำสั่งซื้อ			
Attribute	**Description**	**Data Type**	**Constraint**
order_id	รหัสคำสั่งซื้อ	INT	Foreign Key
promotion_id	รหัสโปรโมชัน	INT	Foreign Key

Payment - การชำระเงิน			
Attribute	**Description**	**Data Type**	**Constraint**
payment_id	รหัสการชำระเงิน	INT	Primary Key

order_id	รหัสคำสั่งซื้อที่ชำระ	TEXT	Foreign Key
employee_id	รหัสพนักงานที่รับชำระ	TEXT	Foreign Key
payment_method	วิธีชำระ เช่น “เงินสด”, “QR”	TEXT	
payment_date	วันที่ชำระเงิน	DATETIME	
payment_amount	จำนวนเงินที่ชำระ	INT	

Promotion - โปรโมชัน			
Attribute	**Description**	**Data Type**	**Constraint**
promotion_id	รหัสโปรโมชัน	INT	Primary Key
promotion_name	ชื่อโปรโมชัน เช่น “ซื้อ 1 แถม 1”, “ลด 10%”	TEXT	
discount_type	ประเภทส่วนลด เช่น “เปอร์เซ็นต์” หรือ “จำนวนเงิน”	TEXT	
discount_value	มูลค่าที่ลด	INT	
start_date	วันที่เริ่มใช้โปรโมชัน	DATE	
end_date	วันที่สิ้นสุดโปรโมชัน	DATE	
active	สถานะการใช้งาน (1=ใช้งานอยู่, 0=หมดอายุหรือปิดการใช้งาน)	INT	

เทคนิคและวิธีการใชนการสร้างองค์ประกอบ

ฟังก์ชั้นการทำงาน

เพิ่มเข้าตะกร้า และ จัดการสินค้าในตะกร้า (พนักงานแคชเชียร์ และ ลูกค้า)

```
document.querySelectorAll(".cart-plus").forEach(btn => {
    btn.addEventListener("click", async () => {
      const li = btn.closest("li");
      const menuId = li.dataset.menuId;

      const res = await fetch("/update-cart", {
        method: "POST",
        headers: {"Content-Type":"application/json"},
        body: JSON.stringify({ menu_id: menuId, change: +1 })
      });

      const data = await res.json();
      if (data.cartHtml) {
        document.querySelector(".container_2").innerHTML = data.cartHtml;
        bindCartEvents(); // rebind ใหม่หลัง render
      }
    });
  });
```

```
document.querySelectorAll(".cart-minus").forEach(btn => {
    btn.addEventListener("click", async () => {
      const li = btn.closest("li");
      const menuId = li.dataset.menuId;

      const res = await fetch("/update-cart", {
        method: "POST",
        headers: {"Content-Type":"application/json"},
        body: JSON.stringify({ menu_id: menuId, change: -1 })
      });

      const data = await res.json();
      if (data.cartHtml) {
        document.querySelector(".container_2").innerHTML = data.cartHtml;
        bindCartEvents(); // rebind ใหม่หลัง render
      }
    });
  });
```

จัดการปุ่ม “+” และ “-” ของสินค้าที่อยู่ในตะกร้า เมื่อผู้ใช้คลิกปุ่ม ระบบจะดึงรหัสสินค้า (menuId) ของรายการนั้น แล้วส่งข้อมูลไปยัง server เพื่อเพิ่มจำนวนสินค้าในตะกร้า หลังจาก server

อัปเดตตะกร้าเสร็จ ระบบจะนำข้อมูลตะกร้าที่อัปเดตกลับมาแสดงบนหน้าเว็บทันที ทำให้ผู้ใช้เห็นจำนวนสินค้าและยอดรวมล่าสุดโดยไม่ต้องโหลดหน้าเว็บใหม่ การเรียก bindCartEvents() ใหม่หลังจากแสดงผล HTML เป็นการทำให้ปุ่มยังคงใช้งานได้หลังจากตะกร้าถูกอัปเดต

```
55  function updateDisplay() {
56      let totalExtra = Object.values(selectedOptions).reduce((sum, opt) => sum + opt.extra, 0);
57      let totalPrice = (parseFloat(priceEl.dataset.basePrice) + totalExtra) * quantity;
58      priceEl.innerText = totalPrice + " ฿";
59
60      qtyInput.value = quantity;
61      optionsInput.value = JSON.stringify(Object.values(selectedOptions));
62      totalPriceInput.value = totalPrice;
63    }
```

ฟอร์มเพิ่มสินค้าลงตะกร้า เมื่อผู้ใช้เลือกตัวเลือกหรือปรับจำนวนสินค้า ระบบจะคำนวณราคาสุทธิ (totalPrice) แบบเรียลไทม์ ฟังก์ชัน updateDisplay() จะรวมราคาฐานกับค่าเสริมของตัวเลือกทั้งหมด คูณกับจำนวนสินค้า แล้วอัปเดตหน้าจอและค่าในฟอร์มให้ตรงกับข้อมูลที่ผู้ใช้เลือก

```
66  form.querySelector(".qty-btn.plus")?.addEventListener("click", () => {
67      quantity++;
68      qtyEl.innerText = quantity;
69      updateDisplay();
70    });
71
72    form.querySelector(".qty-btn.minus")?.addEventListener("click", () => {
73      if (quantity > 1) {
74        quantity--;
75        qtyEl.innerText = quantity;
76        updateDisplay();
77      }
78    });
```

ผู้ใช้สามารถปรับจำนวนสินค้าผ่านปุ่ม “+” และ “-” ซึ่งจะเพิ่มหรือลดค่า quantity และเรียก updateDisplay() เพื่อคำนวณราคาใหม่ทันที นอกจากนี้ การเลือกตัวเลือกแบบ radio จะถูกบันทึกใน selectedOptions และอัปเดตราคาแบบเรียลไทม์เช่นกัน

```
96  form.addEventListener("submit", async (e) => {
97      e.preventDefault();
98
99      const payload = {
100       menu_id: menuId,
101       options: Object.values(selectedOptions),
102       quantity,
103       totalPrice: parseFloat(totalPriceInput.value)
104     };
105
106     const res = await fetch("/add-to-cart", {
107       method: "POST",
108       headers: {"Content-Type":"application/json"},
109       body: JSON.stringify(payload)
110     });
111
112     const data = await res.json();
113
114     if (data.cartHtml) {
115       document.querySelector(".container_2").innerHTML = data.cartHtml;
116       bindCartEvents();
117     }
118
```

เมื่อผู้ใช้กด submit ฟอร์ม ระบบจะรวมข้อมูล menu_id, ตัวเลือก, จำนวน และราคาสุทธิ แล้วส่งไปยัง server ผ่าน /add-to-cart หลังจาก server ส่งตะกร้าที่อัปเดตกลับมา ฟังก์ชันจะ render HTML ตะกร้าใหม่บนหน้าเว็บ และเรียก bindCartEvents()
อีกครั้งเพื่อให้ปุ่มจำนวนสินค้าในตะกร้ายังคงทำงานได้

```
27  function searchProduct() {
28    const searchValue = searchInput.value.toLowerCase();
29    const activeCategory = document.querySelector(".category-btn.active").dataset.category;
30    const visibleSection = activeCategory === "drinks" ? drinksSection : bakerySection;
31
32    const products = visibleSection.querySelectorAll(".product-card, .no-products");
33
34    products.forEach((item) => {
35      const name = item.querySelector(".product-name")?.textContent.toLowerCase() || "";
36      if (name.includes(searchValue) || item.classList.contains("no-products")) {
37        item.style.display = "";
38      } else {
39        item.style.display = "none";
40      }
41    });
42  }
```

ทำหน้าที่ค้นหาสินค้าตามข้อความที่ผู้ใช้พิมพ์ในช่องค้นหา
โดยเริ่มจากแปลงข้อความค้นหาเป็นตัวพิมพ์เล็ก และตรวจสอบ section ของสินค้าที่กำลังแสดงอยู่ (เครื่องดื่มหรือขนมอบ) จากนั้นวนลูปตรวจสอบทุกสินค้า หากชื่อสินค้าตรงกับข้อความค้นหาหรือเป็น element แสดงข้อความว่าไม่มีสินค้า (no-products) ระบบจะคงการแสดงผลไว้ แต่หากไม่ตรงจะซ่อนสินค้านั้น ทำให้ผู้ใช้เห็นเฉพาะสินค้าที่ตรงกับคำค้นหา



```
49  // Helper: แปลงค่าทุกแบบให้เป็น boolean
50  function toBool(val) {
51    return val === true || val === 'true' || val === 1 || val === '1';
52  }
53
54  // Helper: escape HTML ป้องกัน XSS (เช่นชื่อเมนูมี < > " ')
55  function escapeHtml(str) {
56    return String(str)
57      .replace(/&/g, '&amp;')
58      .replace(/</g, '&lt;')
59      .replace(/>/g, '&gt;')
60      .replace(/"/g, '&quot;')
61      .replace(/'/g, '&#39;');
62  }
```

ใช้เพื่อช่วยแปลงค่าต่าง ๆ ให้เป็น boolean และป้องกันปัญหา XSS เมื่อแสดงชื่อเมนูที่มีสัญลักษณ์พิเศษ

```
async function selectItem(id, name, category, isAvailable, stock_qty) {
  isAvailable = toBool(isAvailable);
  selectedItem = { id, name, category, isAvailable, stock_qty };

  const rightPanel = document.getElementById('selectedItems');
  let html = `<h3>${escapeHtml(name)}</h3><hr>`;

  if (category === 'drink') {
    html += `
      <button
        class="btn btn-bg"
        data-id="${id}"
        onclick="toggleDrinkStatus(${id}, ${isAvailable})">
        ${isAvailable ? 'Close Sale' : 'Open Sale'}
      </button>
    `;
  } else if (category === 'bakery') {
    try {
      // 🔹 ดึงข้อมูล stock จากฐานข้อมูล
      const res = await fetch(`/api/stock/${id}`);
      const data = await res.json();
      const dbStock = data.stock_qty ?? 0;

      html += `
  <h5><strong>Stock:</strong> <span id="stock-${id}">${dbStock}</span></h5>
  <div class="stock-controls">
    <div class="d-flex gap-2 mb-2 mt-5">
      <button class="btn btn-sm btn-long flex-fill" onclick="updateStock(${id}, -1)">−</button>
      <button class="btn btn-sm btn-long flex-fill" onclick="updateStock(${id}, 1)">+</button>
      <button class="btn btn-sm btn-bg flex-fill" onclick="editStock(${id})">Edit</button>
    </div>
  </div>
`;

      // <button class="btn btn-bg btn-lg w-100 mt-4" onclick="saveStock(${id})">
      //   Save
      // </button>

    } catch (err) {
      console.error('Error loading stock:', err);
      html += `<p><strong>Stock:</strong> <span style="color:red;">Error</span></p>`;
    }
  }

  rightPane
```

ทำหน้าที่แสดงข้อมูลของสินค้าที่ผู้ใช้เลือก เช่น ชื่อ หมวดหมู่ สถานะเปิด/ปิดการขาย และจำนวนสต็อก หากเป็นเครื่องดื่ม ระบบจะสร้างปุ่มให้เปิดหรือปิดการขายเรียก toggleDrinkStatus() เมื่อเป็นขนมอบ ระบบจะดึงข้อมูลสต็อกจาก backend ผ่าน /api/stock/:id และแสดงปุ่มเพิ่ม ลด หรือแก้ไขสต็อกบน panel ด้านขวา

```
function toggleDrinkStatus(id, currentStatus) {
  // console.log("btn click");

  const newStatus = !currentStatus;
  fetch(`/inventory/toggle/${id}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ is_available: newStatus })
  })
    .then(res => res.json())
    .then(data => {
      if (data.success) {
        const card = document.querySelector(`.product-card[data-id="${id}"]`);
        if (card) card.classList.toggle('unavailable', !data.is_available);
        selectItem(id, selectedItem.name, 'drink', data.is_available, selectedItem.stockQty);
      } else {
        alert('Update failed: ' + (data.message || 'Unknown'));
      }
    })
    .catch(err => {
      console.error('Error toggling drink status:', err);
    })
    .finally(() => {
      window.location.reload();
    })
}

```

ใช้สำหรับเปลี่ยนสถานะเปิด/ปิดการขายของเครื่องดื่ม โดยจะส่งข้อมูลไปยัง endpoint /inventory/toggle/:id และเมื่อ server ตอบกลับสำเร็จ ระบบจะอัปเดต class ของ card เพื่อบ่งบอกว่าสินค้าเปิดหรือปิดการขาย พร้อมเรียก selectItem() อีกครั้งเพื่อนำข้อมูลล่าสุดมาแสดง

```
async function updateStock(id, change) {
  try {
    const res = await fetch(`/inventory/stock/${id}`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ change })
    });

    const data = await res.json();
    if (data.newStock <= 1) {
      window.location.reload();
    }

    if (!data.success) {
      alert(data.message);
    }

    const stockElem = document.getElementById(`stock-${id}`);
    stockElem.textContent = data.newStock;

    // 🔷 อัปเดต class ตาม stock ใหม่
    const card = document.querySelector(`.product-card[data-id="${id}"]`);
    if (card) {
      if (data.newStock <= 0) {
        card.classList.add('unavailable');
      } else {
        card.classList.remove('unavailable');
      }
    }

    res.json({ success: true });

  } catch (err) {
    console.error('Update stock error:', err);
  }

}
```

ใช้สำหรับเพิ่มหรือลดจำนวนสต็อกของสินค้าที่เป็นขนมอบ โดยจะส่งค่า change ไปยัง backend ผ่าน /inventory/stock/:id หลังจาก server อัปเดตสต็อก ระบบจะอัปเดตจำนวนสต็อกบนหน้าเว็บทันที และปรับ class ของ card หากสต็อกหมดหรือมีสินค้าให้แสดง

```
function editStock(id) {
  const current = parseInt(document.getElementById(`stock-${id}`).innerText);
  const newStock = parseInt(prompt("Enter new stock quantity:", current));
  if (isNaN(newStock) || newStock < 0) return;

  const delta = newStock - current;
  updateStock(id, delta);
}
```

ช่วยให้ผู้ดูแลระบบแก้ไขจำนวนสต็อกโดยกรอกตัวเลขเอง ระบบจะคำนวณส่วนต่าง (delta) แล้วเรียก updateStock() เพื่ออัปเดตสต็อกให้ตรงกับค่าที่กรอก

```
192  function clearSelection() {
193    selectedItem = null;
194    document.getElementById('selectedItems').innerHTML = `
195          <div class="empty-state">
196              <p>Please Select Products</p>
197          </div>`;
198  }
```

ใช้สำหรับล้างการเลือกสินค้าใน panel ด้านขวา ทำให้กลับไปแสดงข้อความให้เลือกสินค้าหากยังไม่มีสินค้าที่ถูกเลือก

```
201  async function saveStock(id) {
202    const currentStock = parseInt(document.getElementById(`stock-${id}`).innerText);
203
204    try {
205      const res = await fetch(`/inventory/stock/${id}`, {
206        method: 'POST',
207        headers: { 'Content-Type': 'application/json' },
208        body: JSON.stringify({ change: 0 }) // ไม่เปลี่ยนค่า แค่บันทึก
209      });
210
211      const data = await res.json();
212
213      if (data.success) {
214        alert(`✅ Stock saved successfully!\n\nCurrent stock: ${currentStock}`);
215      } else {
216        alert(`❌ Save failed: ${data.message || "Unknown error"}`);
217      }
218    } catch (err) {
219      console.error('Save stock error:', err);
220      alert('⚠️ Error saving stock. Please try again.');
221    }
222  }
```

ใช้บันทึกสต็อกของสินค้าลง backend โดยไม่เปลี่ยนจำนวนสินค้า (change = 0) หากบันทึกสำเร็จจะแสดง alert ยืนยันให้ผู้ใช้ทราบ

การทำงานของระบบ

หน้าหลักของระบบ

```
document.addEventListener('DOMContentLoaded', () => {
    const sections = document.querySelectorAll('section');
    const navLinks = document.querySelectorAll('header nav a');
    const header = document.querySelector('header');

    function debounce(fn, delay=20) {
        let timer;
        return function(...args){
            clearTimeout(timer);
            timer = setTimeout(() => fn.apply(this, args), delay);
        }
    }

    function scrollSpy() {
        const scrollY = window.scrollY;
        let currentSectionId = sections[0].id;

        sections.forEach(sec => {
            const top = sec.offsetTop - header.offsetHeight - 1; // ใช้ offsetTop แทน rect
            const bottom = top + sec.offsetHeight;
            if(scrollY >= top && scrollY < bottom){
                currentSectionId = sec.id;
            }
        });

        navLinks.forEach(link => {
            const href = link.getAttribute('href').replace('#','');
            link.classList.toggle('active', href === currentSectionId);
        });
    }

    window.addEventListener('scroll', debounce(scrollSpy, 20));
    scrollSpy(); // เรียกตอนโหลด
});
```

สร้างระบบ scroll spy และ highlight เมนูอัตโนมัติเมื่อเลื่อนหน้าเว็บ เมื่อหน้าเว็บโหลดเสร็จ (DOMContentLoaded) ระบบจะดึงทุก section ของหน้าเว็บ (sections), ลิงก์ในเมนูนำทาง (navLinks) และ header (header) เพื่อใช้คำนวณตำแหน่งของแต่ละ section ฟังก์ชัน debounce(fn, delay) ทำหน้าที่ลดความถี่การเรียกฟังก์ชัน โดยรอให้ผู้ใช้หยุดเลื่อนสักครู่ก่อนเรียกฟังก์ชัน scrollSpy() ซึ่งช่วยลดภาระการทำงานของ browser

ฟังก์ชัน scrollSpy() จะตรวจสอบตำแหน่ง scroll ปัจจุบัน (scrollY) และวนลูปทุก section เพื่อหา section ที่กำลังแสดงบนหน้าจอ จากนั้นจะอัปเดต class active ให้กับลิงก์เมนูที่ตรงกับ section นั้น ทำให้เมนูนำทางไฮไลต์ตาม section ที่ผู้ใช้กำลังดู

สุดท้าย ระบบจะผูกฟังก์ชัน scrollSpy() กับ event scroll ของหน้าต่าง โดยใช้ debounce และเรียก scrollSpy() ตอนโหลดหน้าเพื่อให้เมนูไฮไลต์ตรงกับ section เริ่มต้น

การจัดการ modal ของเมนูต่างๆ

```
6  openModalButtons.forEach(button => {
7      button.addEventListener('click', () => {
8          const modal = document.querySelector(button.dataset.modalTarget);
9          openModal(modal);
10     });
11 });
```

เปิด modal เมื่อผู้ใช้คลิกปุ่ม โดยเริ่มจากเลือกปุ่มทั้งหมดที่มี attribute data-modal-target จากนั้นผูก event listener ให้แต่ละปุ่ม เมื่อผู้ใช้คลิก ระบบจะดึง selector ของ modal จาก button.dataset.modalTarget แล้วเรียกฟังก์ชัน openModal(modal) เพื่อเพิ่ม class active ให้กับ modal และ overlay ทำให้ modal ปรากฏบนหน้าเว็บ

ปิด modal เมื่อผู้ใช้คลิกบริเวณ overlay โดยเริ่มจากฟัง event click บน element overlay เมื่อคลิก ระบบจะหา modal ทั้งหมดที่มี class active อยู่ (.modal.active) และเรียกฟังก์ชัน closeModal(modal) กับแต่ละ modal เพื่อเอา class active ออกจาก modal และ overlay ทำให้ modal หายไปจากหน้าจอ

```
14 overlay.addEventListener('click', () => {
15     const modals = document.querySelectorAll('.modal.active');
16     modals.forEach(modal => {
17         closeModal(modal);
18     });
19 });
20
```

ปิด modal เมื่อผู้ใช้คลิกบริเวณ overlay โดยเริ่มจากฟัง event click บน element overlay เมื่อคลิก ระบบจะหา modal ทั้งหมดที่มี class active อยู่ (.modal.active) และเรียกฟังก์ชัน closeModal(modal) กับแต่ละ modal เพื่อเอา class active ออกจาก modal และ overlay ทำให้ modal หายไปจากหน้าจอ

```
22  closeModalButtons.forEach(button => {
23      button.addEventListener('click', () => {
24          const modal = button.closest('.modal');
25          closeModal(modal);
26      });
27  });
```

ปิด modal เมื่อผู้ใช้กดปุ่ม × โดยเริ่มจากเลือกปุ่มทั้งหมดที่มี attribute data-close-button แล้วผูก event listener ให้แต่ละปุ่ม เมื่อผู้ใช้คลิก ระบบจะหา modal ที่เป็น parent ของปุ่มนั้น (button.closest('.modal')) และเรียกฟังก์ชัน closeModal(modal) เพื่อเอา class active ออกจาก modal และ overlay ทำให้ modal หายไปจากหน้าจอ

```
29  function openModal(modal) {
30      if (modal == null) return;
31      modal.classList.add('active');
32      overlay.classList.add('active');
33
34  }
35
36  function closeModal(modal) {
37      if (modal == null) return;
38      modal.classList.remove('active');
39      overlay.classList.remove('active');
40
41
42  }
```

ฟังก์ชัน openModal(modal) จะตรวจสอบก่อนว่ามี modal ที่ส่งเข้ามาจริงหรือไม่ หากมี จะเพิ่ม class active ให้กับ modal และ overlay ทำให้ modal ปรากฏบนหน้าเว็บพร้อมพื้นที่มืดรอบ ๆ

ฟังก์ชัน closeModal(modal) จะตรวจสอบ modal เช่นเดียวกัน จากนั้นลบ class active ทั้งจาก modal และ overlay ทำให้ modal หายไปจากหน้าจอ

```
45  document.querySelectorAll('.add-to-cart').forEach(button => {
46      button.addEventListener('click', (e) => {
47          const form = button.closest("form");
48
49          // ถ้า input ที่เป็น required ยังไม่ได้เลือก
50          if (!form.checkValidity()) {
51              e.preventDefault();       // กันไม่ให้ submit
52              form.reportValidity();    // โชว์ข้อความเตือน browser
53              return;                   // ออกจาก function ไม่ปิด modal
54          }
55
56          // ถ้า valid แล้ว (เลือกครบ)
57          const modal = button.closest('.modal');
58          closeModal(modal);
59      });
60  });
61
```

ปิด modal อัตโนมัติเมื่อผู้ใช้กดปุ่ม “เพิ่มเข้าตะกร้า”ระบบจะเลือกปุ่มทั้งหมดที่มีคลาส .add-to-cart และผูก event listener ให้แต่ละปุ่ม เมื่อผู้ใช้คลิก ระบบจะหา form ที่อยู่ใกล้กับปุ่มนั้น (button.closest("form")) และตรวจสอบความถูกต้องของฟอร์มด้วย checkValidity() ถ้า form ยังไม่ถูกกรอกครบหรือมี input required ที่ไม่ได้เลือก ระบบจะป้องกันไม่ให้ส่งฟอร์ม (preventDefault()), แสดงข้อความเตือนของ browser (reportValidity()), และ ไม่ปิด modal ถ้า form ถูกต้องครบถ้วน ระบบจะหา modal ที่อยู่ใกล้กับปุ่มนั้น (button.closest('.modal')) และเรียกฟังก์ชัน closeModal(modal) เพื่อปิด modal ทำให้ผู้ใช้เห็นว่าการเพิ่มสินค้าสำเร็จและ modal หายไปทันที

ลูกค้าและพนักงานแคชเชียร์

หน้าเริ่มต้นการใช้งาน

```
app.get('/', (req, res) => {
    console.log('Web starting');
    const sql = `SELECT category_id FROM Category ORDER BY category_id ASC LIMIT 1;`;

    db.get(sql, (err, row) => {
        if (err) {
            console.log(err.message);
            // console.log('/ not WORK!!!!')

            return res.send("Error loading categories");
        }
        if (row) {
            res.redirect(`/where`);
            // console.log('/ WORK!!!!')

        } else {
          // ดักว่าไม่มีหมวดหมู่ เพื่อไม่ให้เกิด error
            res.send("No categories found");
            // console.log('/ WORK!!!!')

        }
    });
});
```

เป็นหน้าหลักเพื่อเริ่มเข้าสู่การใช้งาน (บรรทัดที่ 96) เพื่อที่สามารถคลิกเพื่อเริ่มต้นการใช้งาน ทำการดัก error

โดยถ้ามีข้อผิดพลาดในการ(บรรทัดที่ 101-106) query (เช่น ตารางไม่มีอยู่) จะแสดงข้อความ error ใน console และส่งข้อความ "Error loading categories" กลับไปที่ browser หรือ ถ้าผลลัพธ์เป็น undefined (บรรทัดที่ 111 - 113)(ไม่มี row เลย) แสดงข้อความ "No categories found"

การเลือกรับประทาน

```
app.get('/where' ,(req,res) =>{
  res.render('where_to_eat');
});
```

เป็นส่วนที่ต่อมาจากหน้าแรก เมื่อทำการเริ่มต้นเพื่อเริ่มต้นการใช้งาน ระบบจะส่งไปที่หน้า ejs หรือหน้าหลักเพื่อให้ลูกค้าเลือกว่าลูกค้าจะรับประทานที่นี่ หรือ รับกลับบ้าน

```
871  app.get('/main/:where' , (req,res) =>{
872    req.session.where = req.params.where;
873    console.log(req.session.where)
874    res.redirect(`/main_for_customer`);
875  })
```

และเมื่อทำการเลือกว่าจะรับประทานอาหารที่ไหนได้แล้วก็จะมาหน้านี้ เพื่อทำการบันทึกลง session ว่าลูกค้าได้เลือกรับประทานที่ได้ และก็ไปสู่หน้าการใช้งานหลักของลูกค้า

หน้าหลักการใช้งานของลูกค้า (main)

```
749  app.get('/main_for_customer', (req, res) => {
```

ในบรรทัดที่ 749 จะเป็นการเริ่มต้นการทำงานหลักของโค้ด โดยมีลำดับขั้นตอนดังนี้

```
750  const cart = req.session.cart || [];
751  const total = req.session.total || 0;
```

จะดึงค่าตัวแปรจาก session มาใช้งาน โดยถ้าใน session.cart มีข้อมูล ระบบจะนำค่ามาเก็บไว้ในตัวแปร cart แต่ถ้าไม่มีจะกำหนดให้เป็นอาเรย์ว่าง ([ ]) ส่วน session.total ก็เช่นเดียวกัน ถ้ามีค่าใน session จะนำมาเก็บในตัวแปร total แต่ถ้าไม่มีจะกำหนดค่าเริ่มต้นเป็น 0 เพื่อป้องกันข้อผิดพลาดในการทำงานของระบบ.

```
754   const cate_select_sql = `SELECT * FROM Category`;
755   const menu_select_sql = `SELECT * FROM MENU;`;
756   const option_selector_sql = `
757     SELECT
758       m.menu_id,
759       m.menu_name,
760       og.group_name,
761       og.is_required
762     FROM menu m
763     INNER JOIN Menu_OptionGroup mo
764       ON m.menu_id = mo.menu_id
765     INNER JOIN OptionGroup og
766       ON mo.option_group_id = og.option_group_id`;
767
768   const item_selector_sql = `
769    SELECT
770   og.group_name,
771   og.option_group_id,
772   it.option_id,
773   it.option_name,
774   it.extra_price,
775   ioi.ingredient_id -- เพิ่ม ingredient_id
776 FROM OptionGroup og
777 INNER JOIN ItemOption it
778   ON og.option_group_id = it.option_group_id
779 LEFT JOIN ItemOptionIngredient ioi
780   ON it.option_id = ioi.option_id;`;
781
782 const check_if_there_available = `
783   SELECT
784     m.menu_id,
785     me.is_available,
786     m.ingredient_id,
787     m.quantity,
788     i.stock_qty
789   FROM MenuIngredient m
790   INNER JOIN Ingredient i
791     ON m.ingredient_id = i.ingredient_id
792   INNER JOIN Menu me
793     ON me.menu_id = m.menu_id;
794 `;
795   const check_if_there_option_available = `  SELECT
796     mo.menu_id,
797     ioi.ingredient_id,
798     ioi.quantity,
799     i.stock_qty
800
801   FROM Menu_OptionGroup mo
802   NATURAL JOIN ItemOption io
803
804   NATURAL JOIN ItemOptionIngredient ioi
805
806   NATURAL JOIN Ingredient i
807
808   WHERE mo.option_group_id BETWEEN 3 and 5
809 `;
```

ในส่วนนี้เป็นการประกาศคำสั่ง **SQL** เพื่อดึงข้อมูลจากฐานข้อมูล ได้แก่

- cate_select_sql (บรรทัดที่ 754) : ดึงข้อมูลทั้งหมดจากตาราง Category
- menu_select_sql(บรรทัดที่ 755) : ดึงข้อมูลทั้งหมดจากตาราง MENU
- option_selector_sql(บรรทัดที่ 756) : ดึงข้อมูลชื่อเมนู (menu_name) และกลุ่มตัวเลือก (group_name) โดยใช้คำสั่ง **INNER** JOIN เชื่อมโยงข้อมูลระหว่างตาราง MENU,

Menu_OptionGroup และ OptionGroup
เพื่อให้ได้ข้อมูลเมนูพร้อมรายละเอียดของกลุ่มตัวเลือกที่เกี่ยวข้อง.

- item_selector_sql (บรรทัดที่ 768): ดึงข้อมูลชื่อกลุ่มตัวเลือก (group_name), รายการตัวเลือก (option_name), ราคาพิเศษ (extra_price) และรหัสวัตถุดิบ (ingredient_id) โดยเชื่อมตาราง OptionGroup, ItemOption และ ItemOptionIngredient เข้าด้วยกัน

- check_if_there_available (บรรทัดที่ 782):
ใช้ตรวจสอบปริมาณวัตถุดิบคงเหลือของแต่ละเมนู โดยเชื่อมข้อมูลจากตาราง MenuIngredient และ Ingredient เพื่อเปรียบเทียบจำนวนวัตถุดิบที่ต้องใช้กับจำนวนคงคลัง

- check_if_there_option_available (บรรทัดที่ 795):
ใช้ตรวจสอบวัตถุดิบของตัวเลือกเสริมในเมนู โดยเชื่อมข้อมูลจากตาราง Menu_OptionGroup, ItemOption, ItemOptionIngredient และ Ingredient
เพื่อดูว่ายังมีวัตถุดิบเพียงพอหรือไม่ในกลุ่มตัวเลือกที่กำหนด (ในที่นี้คือกลุ่มที่มี option_group_id ระหว่าง 3 ถึง 5). โดยทำการเลือกแค่ option_group_id ที่มีค่า 3-5 เพราะกลุ่มที่ 1และ2 เป็น เป็นการเลือกประเภทเมนู และ ความหวานของเมนู



```
db.all(cate_select_sql, (err, cate) => {
  if (err) return console.log(err.message);
  db.all(menu_select_sql, (err, data_menu) => {
    if (err) return console.log(err.message);
    db.all(option_selector_sql, (err, selector) => {
      if (err) return console.log(err.message);
      db.all(item_selector_sql, (err, item_selector) => {
        if (err) return console.log(err.message);
          db.all(check_if_there_available , (err,check_qty) =>{
            if (err) return console.log(err.message);
            const menuAvailability = {};
            check_qty.forEach(row => {
              const { menu_id, quantity, stock_qty,is_available } = row;
              if (!menuAvailability[menu_id]) menuAvailability[menu_id] = true; // เริ่มต้นให้พร้อมขาย
              if (stock_qty < quantity) {
                menuAvailability[menu_id] = false; // ถ้ามีอันใดไม่พอ → เมนูนี้หมด
              }
              if (is_available == 0){
                menuAvailability[menu_id] = false;
              }
            });
            const data_menu_with_avail = data_menu.map(m => ({
              ...m,
              is_available: menuAvailability[m.menu_id] !== false // ถ้าไม่เจอใน false ถือว่ามีพอ
            }));
          db.all(check_if_there_option_available, (err, check_option) => {
            if (err) return console.log(err.message);

              const optionAvailability = {};
                check_option.forEach(row => {
                  const { menu_id, ingredient_id, quantity, stock_qty } = row;
                  const key = `${menu_id}_${ingredient_id}`;

                  // เริ่มต้นเป็น true ถ้ายังไม่มี key
                  if (!(key in optionAvailability)) optionAvailability[key] = true;

                  // ถ้า stock ไม่พอ → false
                  if (stock_qty < quantity) optionAvailability[key] = false;
                });
```

ในส่วนนี้เป็นกระบวนการดึงข้อมูลจากฐานข้อมูลหลายตารางเพื่อนำไปแสดงผลในหน้าหลักโดยใช้คำสั่ง db.all() ของ SQLite3 เพื่ออ่านข้อมูลทั้งหมดจากแต่ละตาราง
จากนั้นจึงนำข้อมูลมาประมวลผลก่อนส่งไปแสดงผลในหน้าเว็บ ดังนี้

- ดึงข้อมูลจากตาราง Category, Menu, OptionGroup, และ ItemOption
  เพื่อเตรียมข้อมูลหมวดหมู่ เมนู และตัวเลือกต่าง ๆ ของสินค้า

- ใช้คำสั่ง check_if_there_available เพื่อตรวจสอบวัตถุดิบที่ใช้ในแต่ละเมนู
  หากวัตถุดิบไม่เพียงพอ ระบบจะกำหนดให้เมนูนั้นไม่สามารถขายได้
  หรือหากเมนูนั้นถูกปิดทำการขายโดยพนักงานหลังร้าน ก็จะไม่สามารถทำการขายได้
  (is_available = false)

- ใช้คำสั่ง check_if_there_option_available เพื่อตรวจสอบวัตถุดิบของตัวเลือกเสริมในเมนู โดยเปรียบเทียบปริมาณวัตถุดิบที่ต้องใช้กับจำนวนคงคลัง

- เมื่อประมวลผลเสร็จ ระบบจะรวมข้อมูลทั้งหมดและส่งไปยังหน้า main_for_cashier_before

```
853  res.render('main_for_customer', {
854    cart,
855    total,
856    cate,
857    // menu: data_menu,
858    menu: data_menu_with_avail,
859    slt: selector,
860    itslt: item_selector,
861    optionAvailability,
862    activeCateId: 1
```

และทำการส่งข้อมูลที่ได้จากการ ดึงดาต้าเบสไปให้หน้าหลักเพื่อแสดงเมนูที่สามารถขายได้ หรือเมนูที่ทำการเปิดการขาย

หน้าตะกร้าสินค้า

```
702  app.get('/cart' , (req,res) =>{
703    const cart = req.session.cart || [];
704    const total = req.session.total || 0;
705    res.render("cart", { cart, total });
706    // console.log('This is Cart menu',cart)
707  });
```

เป็นหน้าตะกร้าของลูกค้า โดยจะนำ ข้อมูลสินค้าทั้งหมดใน session มาเก็บไว้ที่ตัวแปรต่างๆ โดยที่ตัวแปร cart จะเก็บข้อมูลคำสั่งซื้อ และรายละเอียดทั้งหมด และ total จะเก็บข้อมูลหรือราคาสินค้าทั้งหมด

หน้าตะกร้าสินค้า (การยืนยันสั่งซื้อสินค้า)

```
870  app.get('/confirm_order-customer', (req, res) => {
871    // ถ้าไม่มี cart ให้กลับไปหน้าแรก
872    if (!req.session.cart || req.session.cart.length === 0) {
873      return res.redirect('/main_for_customer');
874    }
875      // ส่งข้อมูลไป render
876    res.render('confirm_order-customer', {
877      cart: req.session.cart,
878      total:  req.session.total,
879      pro: promotion
880    });
881
882  });
```

ในส่วนของยืนยันคำสั่งซื้อในหน้าตะกร้าสินค้า หากในตะกร้าไม่มีสินค้า (บรรทัดที่ 872 – 873) ระบบจะให้ลูกค้ากลับไปเลือกสินค้าใหม่อีกรอบ เพื่อป้องกันการเกิด error แต่ถ้าหากมีสินค้าในตะกร้า ระบบจะส่งข้อมูลสินค้าและราคารวมทั้งหมดของสินค้าไปให้ที่หน้า confirm_order-customer (บรรทัดที่ 876-880)

หน้าตะกร้าสินค้า (การยกเลิกสั่งซื้อสินค้า)

```
883  app.get('/canceled_order-customer', (req, res) => {
884      req.session.cart = [];
885      req.session.total = 0;
886      res.redirect(`/`);
887  });
```

หากต้องการที่จะยกเลิกสินค้า จะทำการ ให้ session ที่เก็บข้อมูลทั้งหมด ว่างเปล่า และราคารวมมีค่าเท่ากับ 0 และก็ให้ย้อนไปหน้าแรกเลย

หน้าชำระเงิน (QR CODE)

```
1058  app.get('/pay_qr-customer', async (req, res) => {
1059    // const amount = 10000000000000000000;
1060  // const amount = req.session.cart.reduce((sum, item) => sum + (item.price * item.quantity), 0);
1061    const amount = req.session.total;
1062    const promptPayID = '0984242409'; // ใส่เบอร์พร้อมเพย์
1063
1064    // ถ้าไม่มี amount ให้ default เป็น 1 บาท
1065    const finalAmount = amount ? parseFloat(amount) : 1;
1066
1067    const payload = generatePayload(promptPayID, { amount: finalAmount });
1068    const qrDataUrl = await qrcode.toDataURL(payload);
1069    // console.log(cart)
1070
1071
1072    res.render('pay_qr-customer',{amount: finalAmount, qr :qrDataUrl})
1073
1074  });
1075
```

จะทำการสร้างตัวแปรมาเก็บข้อมูลราคารวมของสินค้าคือ ตัวแปรamount (บรรทัดที่ 1062) และนำมาแปลงเป็นจำนวนเต็มอีกที และทำการสร้างให้เลือกว่ามีตัวแปร amount หรือไม่ หากไม่มีจะให้ใส่ค่าเงินเข้าไปเป็น 1 บาทก่อน เพื่อกันการ error ของระบบ (บรรทัดที่ 1065) และทำการสร้าง QR code จาก generatePayload (library ที่มีการติดตั้งมาจากภายนอก) โดยจะใส่เป็นเบอร์ของ พร้อมเพย์ และ จำนวนเงิน และทำการสร้างเป็น QR CODE จาก toDataURL (library ที่มีการติดตั้งมาจากภายนอก) และทำการส่งจำนวนราคา และรูปภาพ QR Code ไปแสดงในหน้าหลัก pay_qr-customer

หน้ายืนยันว่าชำระเงินเสร็จสิ้น

```
888  app.get("/cash-customer/:amount", async (req, res) => {
889
```

การทำงาน ของการชำระเงินเสร็จสิ้น (บรรทัดที่ 888 - 1057) มีดังขั้นตอนต่อไปนี้

```
889  try {
890    const cart = req.session.cart || [];
891    const total = req.session.total || 0;
892    const where = req.session.where || 'TAKE AWAY';
893    const amount = parseInt(req.params.amount, 10);
894    const left = amount - total;
895
```

บรรทัดที่ 890 ทำการดึงข้อมูลสินค้าในตะกร้าทั้งหมดมา หรือถ้าหากไม่มีจะเป็น string เปล่า ([ ])เก็บเอาไว้ในตัวแปร cart

บรรทัดที่ 891 ทำการดึงข้อมูลราคาสินค้าในตะกร้าทั้งหมด หรือถ้าหากไม่มีจะเป็น 0 มาเก็บเอาไว้ในตัวแปร total

บรรทัดที่ 892 ทำการดึงข้อมูลว่าลูกค้าต้องการสั่งกลับบ้าน หรือรับประทานที่นี่ มาเก็บไว้ และหากไม่ได้เลือกจะเป็นการนำกลับบ้าน มาเก็บไว้ที่ตัวแปร where

บรรทัดที่ 893 ทำการคำนวณเงินทอน แต่ถ้าหากลูกค้าทำการชำระเงินด้วย QR code เงินทอนจะเป็น 0 มาเก็บไว้ที่ left

```
903  const now = new Date();
904  const datePrefix = now.toISOString().slice(0, 10).replace(/-/g, "");
905
```

สร้าง Order ID โดย หมายเลขออเดอร์ที่ไม่ซ้ำกันในรูปแบบ YYYYMMDD-NNN (เช่น 20251012-001) โดยการสร้างวันนที่ และแปลงเป็นวันที่มาตรฐานISO ("2025-10-12T07:35:00.000Z") โดย toISOString() และทำการตัดเอามาแค่ตัวหน้า 10 ตัวและทำลบ ‘-’ ทุกตัวออกไป

```
907  db.get(
908    `SELECT order_id FROM "Order"
909    WHERE order_id LIKE ?
910    ORDER BY order_id DESC LIMIT 1`,
911    [`${datePrefix}-%`],
912    (err, row) => {
913      if (err) {
914        console.error("❌ Query last order error:", err);
915        return res.status(500).send("เกิดข้อผิดพลาดในการอ่านเลขคำสั่งซื้อ");
916      }
917
918      let nextNumber = 1;
919      if (row && row.order_id) {
920        const lastNum = parseInt(row.order_id.split("-")[1], 10);
921        nextNumber = lastNum + 1;
922      }
923    const orderId = `${datePrefix}-${String(nextNumber).padStart(3, "0")}`;
```

ทำการหาว่าเลขออเดอร์เลขล่าสุดคือเลขอะไร (บรรทัดที่ 907-911) โดนทำการหาจากวันที่วันนั้นและเรียงจากมากไปน้อย และเลือกอันที่มากที่สุดมา 1 ตัว และถ้าหากหาไม่เจอ (บรรทัดที่ 913-916) จะเขียนขึ้นว่าเกิดข้อผิดพลาด และจะต้องทำรายการใหม่ แต่ถ้า เจอ order_id ล่าสุด(บรรทัดที่ 918 – 923) (เช่น 20251012-002) กำหนดให้ order_id ถัดไปเป็น 20251012-003 โดยกำหนดให้ข้อมูลเก็บไว้ สามหลัก ถ้าเลขไม่ถึงสามหลักจะให้เติม 0 เข้าไปแทน แต่ถ้าไม่เจอเลย (วันใหม่) จะให้ เริ่มจาก 1

```
926  db.run(
927    `INSERT INTO 'Order' (order_id, total_price, order_type, status)
928    VALUES (?, ?, ?, 'pending')`,
929    [orderId, total,where],
930    function (err) {
931      if (err) {
932        console.error("❌ Insert order error:", err);
933        return res.status(500).send("บันทึกคำสั่งซื้อไม่สำเร็จ");
934      }
```

บันทึกออเดอร์หลักโดยการเพิ่มข้อมูลหลักของออเดอร์ (Order ID, ราคารวม, ประเภทออเดอร์, สถานะ 'pending') ลงในตาราง Order เพื่อเก็บข้อมูลลงไปไว้ใน Database



```
cart.forEach((item) => {
  db.run(
    `INSERT INTO "OrderItem" (order_id, menu_id, quantity, price)
     VALUES (?, ?, ?, ?)`,
    [orderId, item.menu_id, item.quantity, item.unitPrice],
    function (err2) {
      if (err2) {
        console.error("❌ Insert order item error:", err2);
        return;
      }

      const orderItemId = this.lastID;
      console.log(`เพิ่มเมนู ${item.menu_name} (order_item_id=${orderItemId})`);

      // --- ถ้ามี options ---
      if (item.options && item.options.length > 0) {
        console.log(`🍀 เมนู ${item.menu_name} มี options:`, item.options);

        item.options.forEach((opt) => {
          db.get(
            `SELECT option_id, extra_price FROM ItemOption WHERE option_name = ?`,
            [opt.name],
            (err3, row2) => {
              if (err3) {
                console.error("❌ Error finding option:", err3.message);
                return;
              }

              if (row2) {
                const finalExtra = opt.extra || row2.extra_price || 0;
                db.run(
                  `INSERT INTO "OrderItemOption" (order_item_id, option_id, extra_price)
                   VALUES (?, ?, ?)`,
                  [orderItemId, row2.option_id, finalExtra],
                  (err4) => {
                    if (err4) {
                      console.error("❌ Insert option error:", err4.message);
                    } else {
                      console.log(
                        `เพิ่ม option '${opt.name}' (option_id=${row2.option_id}) extra=${finalExtra}`
                      );
                    }
                  }
                );
```

บันทึกรายการสินค้าและตัดสต็อก (วนลูป บรรทัดที่ 937-976) โดยระบบจะวนลูปสินค้าทุกรายการใน cart และทำการบันทึกรายการ (OrderItem) โดยการเพิ่มของสินค้าแต่ละชิ้น(บรรทัดที่ 938-948) (เช่น ID เมนู, จำนวน, ราคา) ลงในตาราง OrderItem และถ้าหากเมนูนั้นมีตัวเลือกเสริม(บรรทัดที่52-948) จะบันทึกตัวเลือกเสริม (OrderItemOption) โดยเพิ่ม ข้อมูลตัวเลือกเสริมนั้นลงในตาราง OrderItemOption

```
988  const del_stk = `
989    UPDATE Ingredient
990    SET stock_qty = stock_qty - (
991      SELECT mi.quantity * ?  -- จำนวนวัตถุดิบที่ใช้ต่อเมนู × จำนวนที่ลูกค้าสั่ง
992      FROM MenuIngredient mi
993      WHERE mi.ingredient_id = Ingredient.ingredient_id
994        AND mi.menu_id = ?
995    )
996    WHERE ingredient_id IN (
997      SELECT ingredient_id FROM MenuIngredient WHERE menu_id = ?
998    )
999  `;
1000
1001 db.run(del_stk, [item.quantity, item.menu_id, item.menu_id], function (err) {
```

ลดจำนวน stock ของวัตถุดิบตามที่ใช้ในเมนูนั้นใช้ข้อมูลจากตาราง MenuIngredient ที่เก็บ “เมนูนี้ใช้วัตถุดิบอะไร เท่าไหร่”ตัวคูณคือ mi.quantity * item.quantity (วัตถุดิบต่อเมนู × จำนวนที่ลูกค้าสั่ง) อัปเดตเฉพาะวัตถุดิบที่เกี่ยวข้องกับเมนูนั้น

```
1011 const del_stk_option = `
1012   UPDATE Ingredient
1013   SET stock_qty = stock_qty - (
1014     SELECT ioi.quantity * ?
1015     FROM ItemOptionIngredient ioi
1016     WHERE ioi.ingredient_id = Ingredient.ingredient_id
1017       AND ioi.option_id = ?
1018   )
1019   WHERE ingredient_id IN (
1020     SELECT ingredient_id FROM ItemOptionIngredient WHERE option_id = ?
1021   )
1022 `;
1023
1024 db.run(del_stk_option, [item.quantity, item.option_id, item.option_id], function (err2) {
```

สำหรับเมนูที่มี “option” ที่ใช้วัตถุดิบเพิ่ม เช่น “เพิ่มชีส” ก็ต้องลด stock โดย ตาราง ItemOptionIngredient จะเก็บว่าวัตถุดิบไหนใช้ใน option ใดบ้าง เช่น option “เพิ่มชีส” ใช้ “cheese” 1 ชิ้น ถ้าลูกค้าสั่ง 2 แก้ว ก็ลดไป 2 ชิ้น

```
1037 // --- เคลียร์ session หลังบันทึกเสร็จ ---
1038 req.session.cart = [];
1039 req.session.total = 0;
1040
1041 // --- แสดงหน้า payment_success ---
1042 res.render("payment_success-customer", {
1043   total,
1044   left,
1045   orderId,
1046 });
```

และเมื่อทุกอย่างทำการบันทึกลงในฐานข้อมูลแล้วจะทำการล้างข้อมูลที่อยู่ใน session ทั้งหมด และส่งข้อมูลทั้งหมด ไปแสดงหน้า payment_success-customer.ejs พร้อมกับข้อมูลยอดรวม, เงินทอน และ Order ID เพื่อยืนยันว่าการสั่งซื้อเสร็จสมบูรณ์ และสามารถให้ลูกค้าคนอื่นสามารถใช้บริการได้

endpoint สำหรับส่งข้อมูลหรืออัปเดตข้อมูล (สำหรับลูกค้าและแคชเชียร์)

```
604  function recalcCart(session) {
605    session.total = session.cart.reduce(
606      (sum, item) => sum + (item.totalPrice_forMenu || 0),
607      0
608    );
609    return session.cart.reduce((sum, item) => sum + item.quantity, 0);
610  }
```

รวมค่า totalPrice_forMenu ของทุกสินค้า ⟶ เก็บใน session.total และ รวมจำนวนสินค้า (quantity) ของทุกสินค้า และคืนค่า เป็นจำนวนสินค้าทั้งหมด (cartCount)

การอัปเดต ตะกร้าสินค้าเวลากดเพิ่มสินค้าเข้ามาในตะกร้า

```
611  app.post("/add-to-cart", (req, res) => {
612
```

```
612  const { menu_id, quantity, options, unitPrice, totalPrice } = req.body;
613   const qty = Number(quantity) || 1;
614   const uPrice = unitPrice !== undefined
615     ? Number(unitPrice)
616     : totalPrice ? Number(totalPrice) / qty : 0;
617
618   if (!req.session.cart) req.session.cart = [];
```

อ่านค่าทั้งหมดที่ทำการส่งเข้ามา และกำหนดให้ จำนวนสินค้าเป็น 1 หากไม่มีการส่งเข้ามา และกำหนดให้ราคาต่อหน่วยสินค้า และถ้าหากไม่มีการส่งเข้า จะเป็นราคาทั้งหมด หารด้วยจำนวน และถ้าหาก session ตะกร้าไม่มีข้อมูลอยู่จะทำให้เป็น ค่าว่างเปล่า เพื่อไม่ให้เกิด error

```
623  const existingItem = req.session.cart.find(
624      item => item.menu_id === menu_id && JSON.stringify(item.options) === optionsStr
625    );
```

ทำการตรวจหาว่ามีสินค้านั้นอยู่ในตะกร้าก่อนแล้วหรือเปล่า

```
627  if (existingItem) {
628      existingItem.quantity += qty;
629      if (!existingItem.unitPrice || existingItem.unitPrice === 0) {
630        existingItem.unitPrice = uPrice;
631      }
632      existingItem.totalPrice_forMenu = existingItem.unitPrice * existingItem.quantity;
633
634      const cartCount = recalcCart(req.session);
635      console.log("✅ session cart:", req.session.cart);
636      console.log("✅ session total:", req.session.total);
637
638      // render cart partial ส่ง HTML กลับ client
639      return res.render("cart", { cart: req.session.cart, total: req.session.total }, (err, html) => {
640        if (err) return res.status(500).json({ success: false, message: err.message });
641        res.json({ success: true, cartHtml: html, cartCount, total: req.session.total, message: "อัปเดตสินค้าเรียบร้อย!" });
642      });
643    }
```

หากมีสินค้าที่เหมือนกันอยู่ในตะกร้าก่อนหน้าแล้ว (บรรทัดที่ 627-628) จะทำการเพิ่มจำนวนปริมาณเข้าไปแทน ถ้าหากไม่มีสินค้าที่เหมือนกันอยู่ในตะกร้า (บรรทัดที่ 629-641) จะทำการเพิ้มสินค้าเข้าไปในตะกร้าใหม่ และทำการเรียก function recalcCart ใหม่เพื่อเพิ่มสินค้าเข้าไปในตะกร้า และคำนวณราคารวมทั้งหมดใหม่ และทำการส่งผลลัพธ์กลับไปหา client

การอัปเดต ตะกร้าสินค้าเวลากดเพิ่มสินค้าในตะกร้า

```
672  app.post("/update-cart", (req, res) => {
673      const { menu_id, options, quantity } = req.body;
674      const qty = Number(quantity);
675
676      if (!req.session.cart) req.session.cart = [];
677
678      const optionsStr = JSON.stringify(options);
679
680      console.log(optionsStr)
681      const existingItemIndex = req.session.cart.findIndex(
682          item => item.menu_id === menu_id && JSON.stringify(item.options) === optionsStr
683      );
684
685      if (existingItemIndex !== -1) {
686          if (qty > 0) {
687              req.session.cart[existingItemIndex].quantity = qty;
688              req.session.cart[existingItemIndex].totalPrice_forMenu = req.session.cart[existingItemIndex].unitPrice * qty;
689          } else {
690              // ลบ item เมื่อ quantity = 0
691              req.session.cart.splice(existingItemIndex, 1);
692          }
693      }
694
695      // คำนวณ total
696      req.session.total = req.session.cart.reduce((sum, item) => sum + item.totalPrice_forMenu, 0);
697
698      res.json({ success: true, cart: req.session.cart, total: req.session.total });
699      console.log("✅ session cart for updating cart:", req.session.cart);
700      console.log("✅ session total for updating cart:", req.session.total);
701  });
```

ทำการแปลง options เป็น string เพื่อเปรียบเทียบ และ ใช้ findIndex หา index ของสินค้าในตะกร้าที่ตรงกับ menu_id และ options ถ้าหากพบสินค้า (บรรทัดที่ 685 - 693) ถ้าพบสินค้าในตะกร้า และถ้า qty > 0 ⟶ อัปเดต quantity และปรับ totalPrice_forMenu และถ้าหาก qty = 0 จะทำการ ลบสินค้านั้นออกจาก cart และทำการคำนวณยอดรวมใหม่ (บรรทัดที่ 696)

การอัปเดต ราคาสินค้า

```
708  app.post('/update-total', (req, res) => {
709    const { promotion_id, discountedTotal, discountValue } = req.body;
710    req.session.total = discountedTotal;
711    req.session.promotion_id = promotion_id;
712    req.session.discount = discountValue;
713    res.json({ success: true });
714  });
```

ทำการดึงข้อมูลทั้งหมดมา และทำการอัปเดตราคาสินค้าทั้งหมด ไม่ว่าจะเป็น ราคารวม promotion_id และราคาส่วนลด ของโปรโมชั่นต่อ ออเดอร์นั้น

หน้าหลักของ bartender

```
app.get('/bartender', function (req, res) {
    const query = `
        SELECT
            o.order_id,
            time(o.order_time) AS order_time,
            o.order_type,
            json_group_array(
                json_object(
                    'quantity', oi.quantity,
                    'menu_name', m.menu_name,
                    'option_name', i.option_name
                )
            ) AS items
        FROM 'Order' o
        LEFT JOIN OrderItem oi
        ON oi.order_id = o.order_id
        LEFT JOIN Menu m
        ON oi.menu_id = m.menu_id
        LEFT JOIN OrderItemOption oip
        ON oip.order_item_id = oi.order_item_id
        LEFT JOIN ItemOption i ON i.option_id = oip.option_id
        WHERE m.category_id != 7 and o.status = 'pending'
        GROUP BY o.order_id
        ORDER BY o.order_time
    `;

    db.all(query, (err, rows) => {
        if (err) {
            console.log(err.message);
        }
        res.render('main', { data: rows });
    });
});
```

โค้ดส่วนนี้แสดงข้อมูลออเดอร์ที่อยู่ในสถานะรอดำเนินการ (pending) โดยดึงข้อมูลจากฐานข้อมูล SQLite ผ่านคำสั่ง SQL เพื่อรวมข้อมูลรายละเอียดของออเดอร์ เช่น รหัสออเดอร์ (order_id), เวลาที่สั่ง (order_time), ประเภทออเดอร์ (order_type) และอื่นๆ ของแต่ละเมนู จากนั้นจะกรองเฉพาะเมนูเครื่องดื่ม (category_id != 7) และมีสถานะเป็น “pending” เท่านั้น ผลลัพธ์ที่ได้จะถูกจัดกลุ่มตามหมายเลขออเดอร์และเรียงตามเวลา ก่อนจะถูกส่งไปแสดงผลในหน้า main ผ่าน res.render()
เพื่อให้บาร์เทนเดอร์สามารถดูรายการเครื่องดื่มที่ต้องจัดเตรียมได้อย่างชัดเจนและเป็นระบบ.



```
1236  app.get('/api/stock/:id', (req, res) => {
1237    const id = req.params.id;
1238    // console.log(id);
1239    const sql = `SELECT stock_qty FROM ingredient WHERE ingredient_name = (select menu_name from menu where menu_id = ${req.params.id})`;
1240    console.log(sql);
1241
1242    db.get(sql, (err, row) => {
1243
1244      console.log(row)
1245      if (err) return res.status(500).json({ error: err.message });
1246      res.json({ stock_qty: row ? row.stock_qty : 0 });
1247    });
1248  });
```

โค้ดส่วนนี้ทำหน้าที่ในการดึงข้อมูลจำนวนคงเหลือของวัตถุดิบ (Stock) จากฐานข้อมูล โดยรับค่า menu_id มาจาก URL ผ่านพารามิเตอร์ :id จากนั้นระบบ จะทำการหาชื่อเมนู (menu_name) จากตาราง menu โดยใช้ค่า menu_id ที่ได้รับเข้ามา และนำชื่อเมนูที่ได้จากขั้นตอนแรก ไปค้นหาในตาราง ingredient โดยตรวจสอบว่า ingredient_name มีค่าตรงกับชื่อเมนูนั้นหรือไม่ เมื่อพบข้อมูลที่ตรงกัน ระบบจะดึงค่าของ stock_qty (จำนวนคงเหลือในสต็อก) ออกมาส่งผลลัพธ์กลับไปในรูปแบบ JSON โดยมีคีย์เป็น stock_qty ซึ่งแสดงจำนวนคงเหลือของวัตถุดิบในสต็อก หากไม่พบข้อมูลที่ตรงกัน (บรรทัดที่ 1246) ระบบจะส่งค่า 0 กลับไปแทน เพื่อป้องกันไม่ให้เกิดข้อผิดพลาดขณะประมวลผล

ดูสินค้าในคลัง



```
1131  app.get('/inventory_for_barrista', (req, res) => {
1132
```

```
1289        SELECT
1290            o.order_id,
1291            time(o.order_time) AS order_time,
1292            o.order_type,
1293            json_group_array(
1294                json_object(
1295                    'quantity', oi.quantity,
1296                    'menu_name', m.menu_name,
1297                    'option_name', i.option_name
1298                )
1299            ) AS items
1300        FROM 'Order' o
1301        LEFT JOIN OrderItem oi
1302            ON oi.order_id = o.order_id
1303        LEFT JOIN Menu m
1304            ON oi.menu_id = m.menu_id
1305        LEFT JOIN OrderItemOption oip
1306            ON oip.order_item_id = oi.order_item_id
1307        LEFT JOIN ItemOption i
1308            ON i.option_id = oip.option_id
1309        WHERE m.category_id != 7 and o.status = 'pending'
1310        GROUP BY o.order_id
1311        ORDER BY o.order_time
1312    `;
```

โดยทำการดึงข้อมูลจากฐานข้อมูลและทำการ ดึงเฉพาะออเดอร์ที่มีสถานะเป็น 'pending' (ยังไม่เสร็จ) และใช้ฟังก์ชัน json_group_array() และ json_object() เพื่อรวมข้อมูลสินค้าภายในออเดอร์ให้อยู่ในรูปแบบ JSON

```
1315  const menuQuery = `SELECT m.menu_image,m.category_id, m.menu_id, m.menu_name, m.base_price, m.is_available, i.stock_qty
1316                FROM Menu m
1317                LEFT JOIN Ingredient i
1318                  on (m.menu_name = i.ingredient_name)`;
1319  // รัน query ออเดอร์ก่อน
1320  db.all(orderQuery, (err, data) => {
```

ทำการดึงข้อมูลเมนูทั้งหมดจากตาราง Menu พร้อมเชื่อมตาราง Ingredient เพื่อดูจำนวนสต็อก (stock_qty)

```
1339  const format_bakery = bakery.map(d => ({
1340      menu_image: d.menu_image,
1341      menu_id: d.menu_id,
1342      menu_name: d.menu_name,
1343      base_price: d.base_price,
1344      is_available: d.is_available === 1 ? true : false,
1345      stock_qty: d.stock_qty
1346  }));
1347
1348  const format_drink = drinks.map(d => ({
1349      menu_image: d.menu_image,
1350      menu_id: d.menu_id,
1351      menu_name: d.menu_name,
1352      base_price: d.base_price,
1353      is_available: d.is_available === 1 ? true : false,
1354      stock_qty: d.stock_qty
1355  }));
```

ทำการแยหมวดหมู่และทำการจัด format เมนูให้เหมือนกันทั้งหมด

```
1361  res.render('inventory_for_cashier', {
1362        data,   // ออเดอร์ทั้งหมด
1363        drinks : format_drink,   // เมนูเครื่องดื่ม
1364        bakery : format_bakery  // เมนู bakery
1365    });
```

และทำการส่งข้อมูบทั้งหมดไปโชว์ในหน้า inventory_for_cashier เพื่อทำการแสดงข้อมูลทั้งหมด

ดู Stock สินค้าหมวดหมู่เบเกอรี่

```
1390  app.get('/api/stock/:id', (req, res) => {
1391    const id = req.params.id;
1392    // console.log(id);
1393    const sql = `SELECT stock_qty FROM ingredient WHERE ingredient_name = (select menu_name from menu where menu_id = ${req.params.id})`;
1394    console.log(sql);
1395
1396    db.get(sql, (err, row) => {
1397
1398      console.log(row)
1399      if (err) return res.status(500).json({ error: err.message });
1400      res.json({ stock_qty: row ? row.stock_qty : 0 });
1401    });
1402  });
```

ในโค้ดส่วนนี้ใช้ดึงข้อมูลจำนวนคงเหลือของวัตถุดิบจากฐานข้อมูล โดยรับค่า menu_id มาจาก URL ของผู้ใช้งาน จากนั้นระบบจะนำรหัสเมนูที่ได้รับไปค้นหาชื่อเมนู (menu_name) จากตาราง menu แล้วนำชื่อเมนูนั้นไปค้นหาในตาราง ingredient เพื่อหาค่าจำนวนคงเหลือ (stock_qty) ของวัตถุดิบที่มีชื่อเดียวกัน เมื่อพบข้อมูล ระบบจะส่งค่าจำนวนคงเหลือกลับไปในรูปแบบ JSON แต่หากไม่พบข้อมูล ระบบจะส่งค่า 0 กลับไปแทน เพื่อป้องกันไม่ให้เกิดข้อผิดพลาด โค้ดนี้จึงมีหน้าที่หลักในการตรวจสอบจำนวนวัตถุดิบคงเหลือของเมนูแต่ละรายการ และใช้แสดงผลให้กับส่วนอื่นของระบบ เช่น หน้าพนักงานขายหรือหน้าจัดการสต็อก เพื่อให้สามารถตรวจสอบสถานะวัตถุดิบได้แบบเรียลไทม์อย่างสะดวกและรวดเร็ว.

endpoint สำหรับส่งข้อมูลหรืออัปเดตข้อมูล (สำหรับบาริสต้า)

การอัปเดตสถานะคำสั่งซื้อ

```
1118  app.post("/complete-order/:id", (req, res) => {
1119      const orderId = req.params.id;
1120      const sql = `UPDATE 'Order' SET status = 'complete' WHERE order_id = ?`;
1121
1122      db.run(sql, [orderId], function(err) {
1123          if (err) {
1124              console.error(err.message);
1125              return res.json({ success: false, error: err.message });
1126          }
1127          res.json({ success: true });
1128      });
1129  });
```

ทำการนำค่า id มาจาก url ที่ส่งเข้ามา และทำการเปลี่ยนสถานะจากตอนแรกเป็น pending การเปลี่ยนเป็น complete

จัดการสถานะเมนูเครื่องดื่ม

```
app.post('/inventory/toggle/:id', (req, res) => {
  const id = req.params.id;
  const { is_available } = req.body;

  if (typeof is_available === 'undefined') {
    return res.status(400).json({ success: false, message: 'Missing is_available field' });
  }

  const sql = `UPDATE Menu SET is_available = ? WHERE menu_id = ?`;
  // console.log(sql);
  db.all(sql, [is_available ? 1 : 0, id], function (err) {
    if (err) {
      console.error('DB Error:', err);
      return res.status(500).json({ success: false, message: 'Database update failed' });
    }

    res.json({ success: true, is_available });
  });
});
```

จะทำการเปลี่ยนสถานะการวางจำหน่ายของเมนู (เปิด/ปิด) โดยการดึง id ของเมนูจาก URL parameter และทำการเปลี่ยนค่าถ้าหากมันเปิดอยู่ก็จะปิดและทำการบันทึกข้อมูลลงฐานจ้อมูล

จัดการคลังสินค้าของเบเกอรี่

```
app.post('/inventory/stock/:id', (req, res) => {
  const id = req.params.id;
  const { change } = req.body;

  if (typeof change !== 'number') {
    return res.status(400).json({ success: false, message: 'Invalid or missing change value' });
  }


  const sqlGet = `
    SELECT m.menu_id, m.menu_name, i.stock_qty
    FROM Menu m
    JOIN Ingredient i ON i.ingredient_name = m.menu_name
    WHERE m.menu_id = ? AND m.category_id = 7
  `;

  db.get(sqlGet, [id], (err, row) => {
    if (err) return res.status(500).json({ success: false, message: 'Database query failed' });
    if (!row) return res.status(404).json({ success: false, message: 'Bakery not found' });

    const newStock = Math.max(0, row.stock_qty + change);

    const sqlUpdate = `UPDATE Ingredient SET stock_qty = ? WHERE ingredient_name = ?`;
    db.run(sqlUpdate, [newStock, row.menu_name], function (err2) {
      if (err2) return res.status(500).json({ success: false, message: 'Database update failed' , error: err2.message});
      res.json({
        success: true,
        menu_id: row.menu_id,
        menu_name: row.menu_name,
        newStock
      });
    });
  });
});
```

ดึง id ของเมนูจาก URL parameter และอ่านค่า change จาก request body เพื่อบวกหรือลดสต็อก จากนั้นตรวจสอบว่า change เป็นตัวเลขหรือไม่ หากไม่ใช่จะส่ง error กลับ client ต่อมาจะดึงข้อมูลเมนูและจำนวนสต็อกปัจจุบันจากฐานข้อมูลโดยเชื่อม Menu กับ Ingredient

แล้วคำนวณสต็อกใหม่ (newStock) และป้องกันค่าไม่ให้ติดลบ (บรรทัดที่ 1300) จากนั้นอัปเดตค่า stock_qty ในฐานข้อมูลและส่ง JSON กลับ client พร้อมข้อมูลเมนูและสต็อกใหม่

คู่มือการใช้งาน

ลูกค้า

หน้าเริ่มต้นการใช้งาน



โดยหน้าแรกการใช้งาน จะต้องกดหน้าจอ 1 ครั้งเพื่อเริ่มต้นการใช้งาน

เลือกรับประทานที่ไหน



หลังจากแตะหน้าจอ 1 ที ระบบจะเขามาสู่ให้เลือกว่าจะรับประทานอาหารที่ไหน

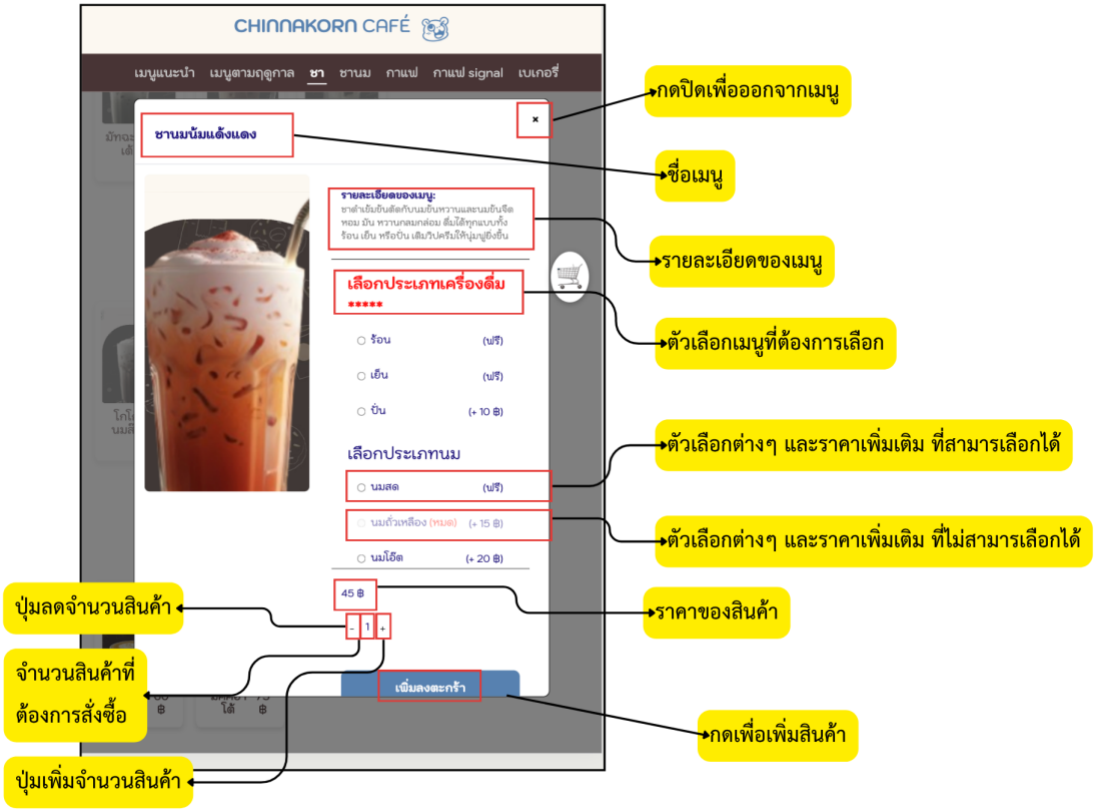
หน้าเลือกเมนู (main_for_customer)



เมื่อผู้ใช้ทำการเลือกสถานที่สำหรับรับประทานอาหารเรียบร้อยแล้ว ระบบจะนำผู้ใช้เข้าสู่ **หน้าหลักสำหรับการเลือกรายการสินค้า** โดยในหน้านี้ผู้ใช้สามารถเลือกเมนูที่ต้องการสั่งซื้อได้อย่างสะดวก ผ่านการเลื่อนดูเมนูทั้งหมด หรือเลือกจาก **หมวดหมู่เมนู** ที่แสดงอยู่บริเวณด้านบนของหน้า เพื่อช่วยให้ค้นหาเมนูที่ต้องการได้รวดเร็วมากยิ่งขึ้น

ในหน้าดังกล่าว ระบบจะแสดงรายการสินค้าพร้อมชื่อเมนู ภาพประกอบ และราคาสินค้าอย่างชัดเจน เพื่อให้ผู้ใช้สามารถตัดสินใจเลือกซื้อได้ง่ายขึ้น ทั้งนี้ หากเมนูใด **หมดสต็อก** หรืออยู่ในสถานะ **ปิดการขาย** ระบบจะแสดงข้อความว่า “หมด” บนเมนูนั้น และจะ **ไม่อนุญาตให้ผู้ใช้ทำการสั่งซื้อ** เมนูดังกล่าว เพื่อป้องกันความผิดพลาดในการสั่งสินค้า

หน้าเลือกเมนูสินค้า



เมื่อผู้ใช้ทำการเลือกรายการสินค้าที่ต้องการสั่งซื้อเรียบร้อยแล้ว ระบบจะนำผู้ใช้เข้าสู่หน้าสำหรับเลือกตัวเลือกเพิ่มเติมของสินค้า (เช่น เครื่องดื่ม) ซึ่งหน้าดังกล่าวจะประกอบไปด้วยส่วนแสดงข้อมูลและองค์ประกอบที่สำคัญ ดังนี้

1. **ชื่อเมนูและรูปภาพสินค้า**

   แสดงชื่อเมนูพร้อมภาพประกอบ เพื่อให้ผู้ใช้สามารถระบุและเข้าใจลักษณะของสินค้าที่เลือกได้อย่างชัดเจน

2. **รายละเอียดสินค้า**

   แสดงข้อมูลเพิ่มเติมเกี่ยวกับสินค้า เช่น ส่วนผสม รสชาติ หรือคุณลักษณะเฉพาะของเมนูนั้น

3. **ตัวเลือกของสินค้า**

   ผู้ใช้สามารถเลือกตัวเลือกเพิ่มเติมที่เกี่ยวข้องกับสินค้า เช่น ระดับความหวาน ชนิดของนม หรือท็อปปิ้งเพิ่มเติม โดยมีรายละเอียดดังนี้

- หากเป็นตัวเลือกที่จำเป็นต้องเลือก ระบบจะแสดง **ตัวอักษรสีแดง** และ **เครื่องหมายดอกจัน (*)** เพื่อระบุความจำเป็นในการเลือก

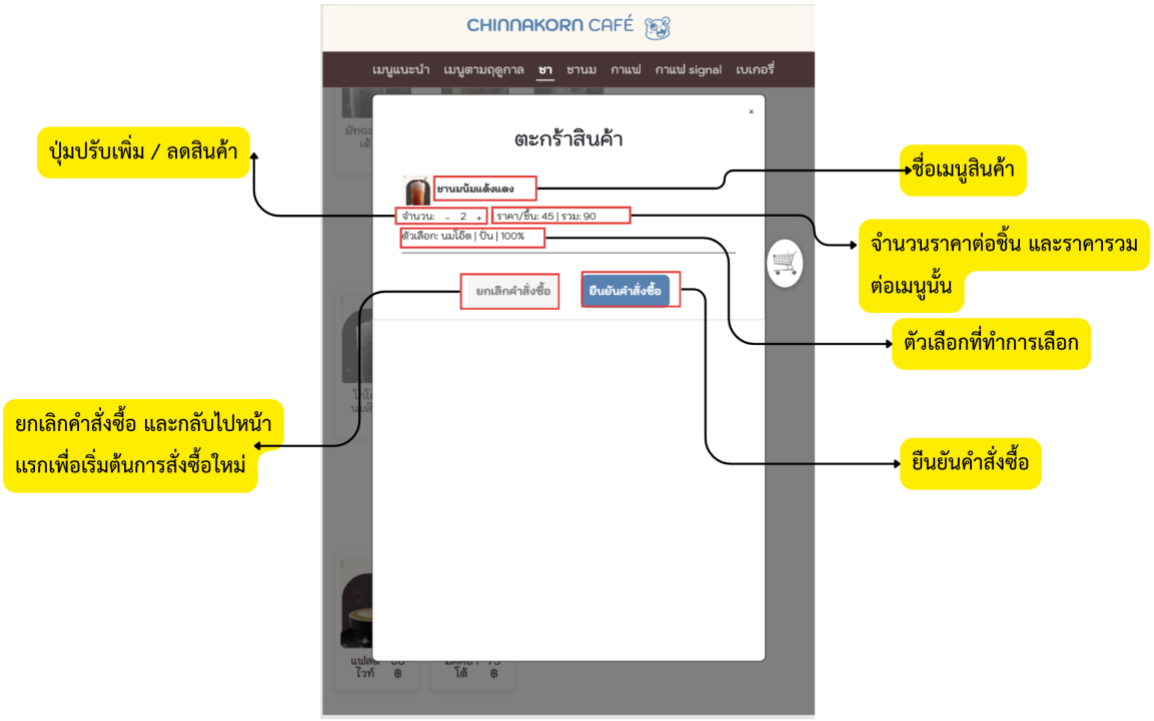
- ในกรณีที่ตัวเลือกใด **หมดสต็อก** ระบบจะแสดงสถานะปิดและไม่อนุญาตให้ผู้ใช้ทำการเลือกได้

- สำหรับตัวเลือกที่มี **ค่าใช้จ่ายเพิ่มเติม** ระบบจะแสดงข้อความกำกับไว้ด้านหลังตัวเลือก เพื่อแจ้งจำนวนเงินที่ต้องชำระเพิ่มอย่างชัดเจน

4. **ส่วนการจัดการคำสั่งซื้อ**

   แสดงจำนวนสินค้า ราคา ปุ่มเพิ่มหรือลดจำนวนสินค้า และปุ่มสำหรับเพิ่มสินค้าลงในตะกร้า
   เพื่อให้ผู้ใช้สามารถปรับแต่งรายการสั่งซื้อได้อย่างสะดวกและถูกต้องก่อนยืนยันการสั่งซื้อ

หน้าตะกร้าสินค้า (cart)



เมื่อผู้ใช้กดเพิ่มรายการสินค้าที่ต้องการ ระบบจะทำการบันทึกรายการสินค้านั้นเข้าสู่ **ตะกร้าสินค้า** โดยผู้ใช้สามารถเข้าสู่หน้าตะกร้าเพื่อดูรายละเอียดของสินค้าที่ได้เลือกไว้ได้ ซึ่งภายในตะกร้าจะแสดงข้อมูลสำคัญดังนี้

1. **ชื่อเมนูและตัวเลือกสินค้า**
   แสดงชื่อเมนูพร้อมรายละเอียดของตัวเลือกที่ผู้ใช้ได้ทำการเลือกไว้สำหรับสินค้านั้น
   เพื่อให้ผู้ใช้สามารถตรวจสอบความถูกต้องของรายการก่อนทำการสั่งซื้อ

2. **ราคาสินค้า**
   แสดงราคาต่อหนึ่งหน่วยของสินค้า รวมถึงราคารวมของสินค้าทั้งหมดในตะกร้า เพื่อให้ผู้ใช้สามารถทราบยอดชำระโดยรวมได้อย่างชัดเจน

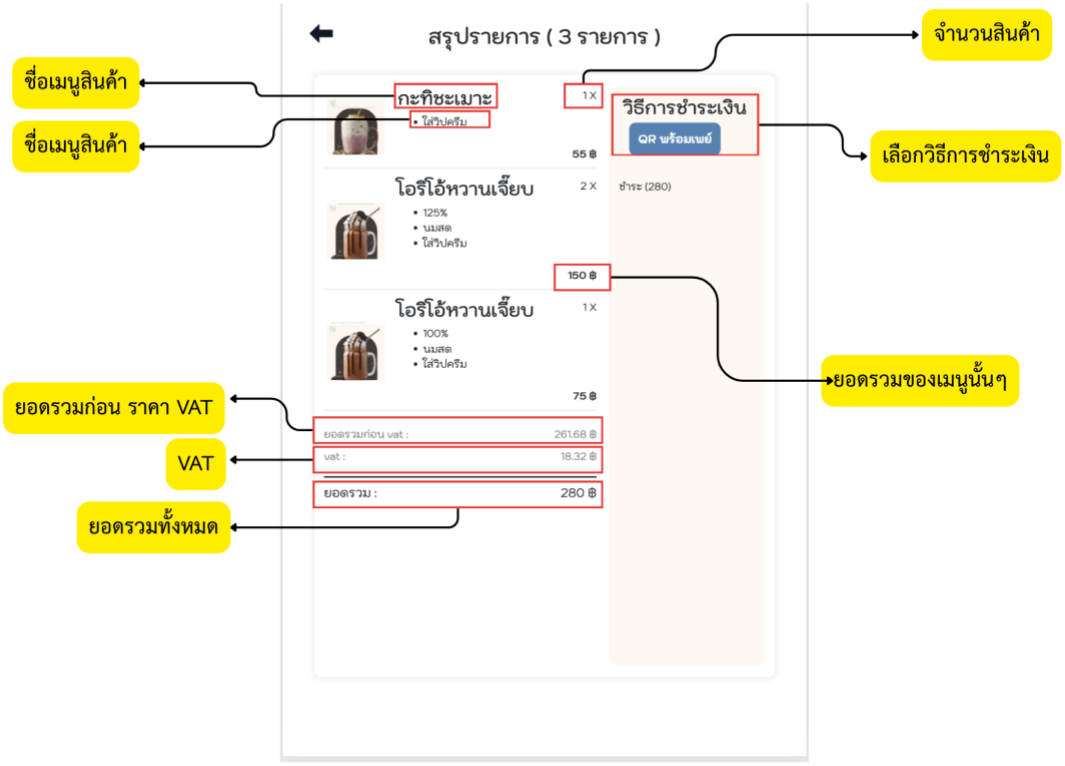
3. **การปรับจำนวนสินค้า**
   ผู้ใช้สามารถเพิ่มหรือลดจำนวนสินค้าภายในตะกร้าได้ตามต้องการ หากผู้ใช้ปรับจำนวนสินค้าลงเป็นศูนย์ (0)
   ระบบจะทำการลบสินค้านั้นออกจากตะกร้าโดยอัตโนมัติ

4. **การยืนยันและยกเลิกคำสั่งซื้อ**

- หากผู้ใช้ต้องการดำเนินการสั่งซื้อ ระบบมีปุ่ม **“ยืนยันคำสั่งซื้อ”** เพื่อเข้าสู่ขั้นตอนถัดไปของการสั่งซื้อสินค้า

- ในกรณีที่ผู้ใช้ไม่ประสงค์จะสั่งซื้อ ระบบมีปุ่ม **“ยกเลิกคำสั่งซื้อ”** เพื่อยกเลิกรายการทั้งหมด
  และระบบจะนำผู้ใช้กลับไปยังหน้าแรกของระบบโดยอัตโนมัติ

หน้ายืนยันคำสั่งซื้อ



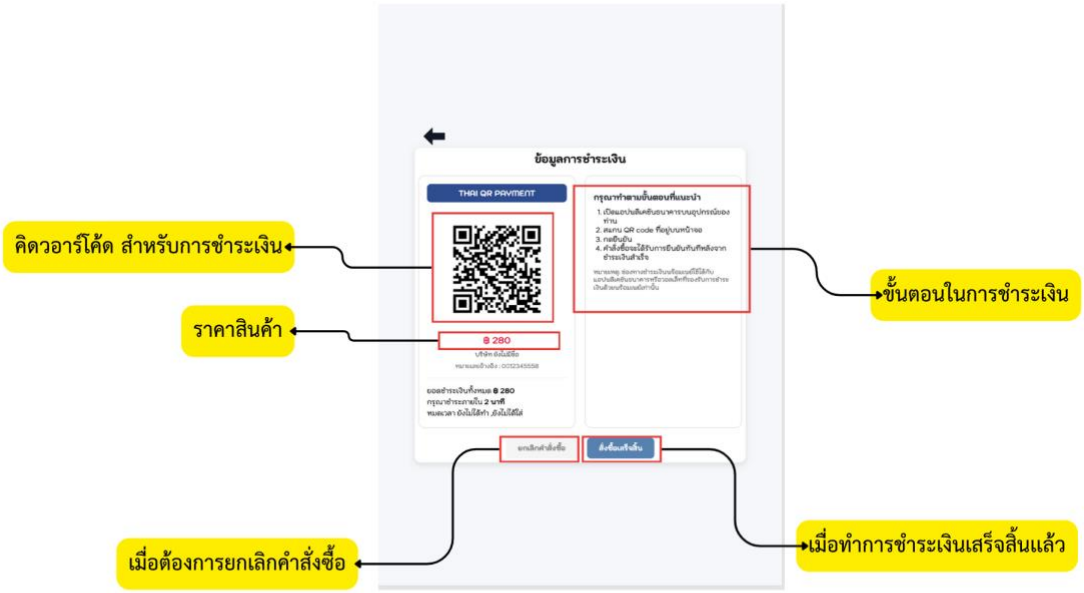
หลังจากผู้ใช้กดยืนยันการสั่งซื้อ ระบบจะนำผู้ใช้เข้าสู่ **หน้าชำระเงิน**
ซึ่งในหน้านี้จะแสดงรายละเอียดของรายการสินค้าที่ผู้ใช้ได้ทำการเลือกไว้ทั้งหมด ประกอบด้วย **ชื่อเมนู ตัวเลือกของเมนู จำนวนสินค้าที่สั่งซื้อ ราคาต่อหน่วย** และ **ราคารวมของแต่ละรายการ** เพื่อให้ผู้ใช้สามารถตรวจสอบความถูกต้องของคำสั่งซื้อได้ก่อนทำการชำระเงิน

นอกจากนี้ ระบบจะแสดง **สรุปรายละเอียดราคาทั้งหมด** ได้แก่

- **ราคารวมก่อนภาษีมูลค่าเพิ่ม (ก่อน VAT)**
- **จำนวนภาษีมูลค่าเพิ่ม (VAT)**
- **ราคารวมสุทธิ (ยอดชำระทั้งหมด)**

ภายในหน้านี้ ผู้ใช้สามารถเลือกวิธีการชำระเงินที่ต้องการได้ โดยในปัจจุบัน ระบบรองรับการชำระเงินในรูปแบบ QR Code Payment ซึ่งเป็นช่องทางการชำระเงินหลักสำหรับลูกค้าในระบบ เพื่อความสะดวกและรวดเร็วในการดำเนินการชำระเงิน

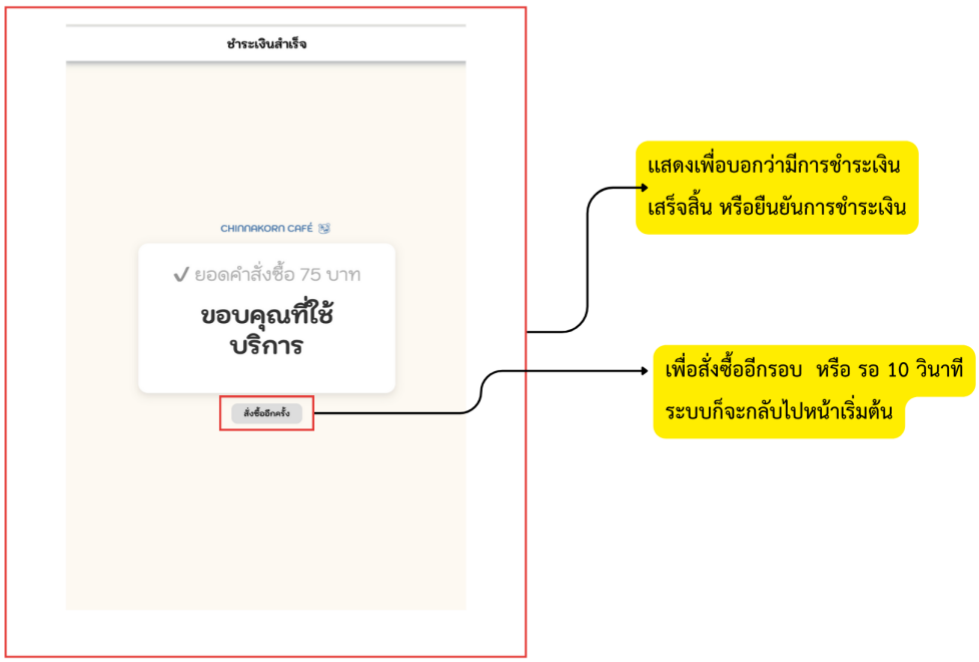
หน้าชำระเงิน



ในหน้าชำระเงิน ระบบจะแสดง QR Code **สำหรับการชำระเงิน** พร้อมทั้ง **รายละเอียดราคาสินค้า** เพื่อให้ผู้ใช้สามารถตรวจสอบความถูกต้องของยอดชำระก่อนดำเนินการชำระเงินจริง นอกจากนี้ ภายในหน้ายังมีการแสดง **ขั้นตอนและวิธีการชำระเงิน** อย่างชัดเจน เพื่อให้ผู้ใช้สามารถทำตามได้อย่างถูกต้องและสะดวกต่อการใช้งาน

เมื่อผู้ใช้ทำการชำระเงินเรียบร้อยแล้ว สามารถกดปุ่ม **“สั่งซื้อเสร็จสิ้น”** เพื่อยืนยันการทำรายการและเข้าสู่ขั้นตอนถัดไปของระบบได้ทันที ในกรณีที่เกิดปัญหาระหว่างการทำรายการ หรือไม่สามารถดำเนินการชำระเงินได้ ผู้ใช้สามารถกดปุ่ม **“ยกเลิกคำสั่งซื้อ”** เพื่อยกเลิกการสั่งซื้อทั้งหมดได้ โดยระบบจะยกเลิกคำสั่งซื้อและนำผู้ใช้กลับไปยังหน้าแรกโดยอัตโนมัติ

หน้ายืนยันการชำระเงิน หรือ ชำระเงินสำเร็จ

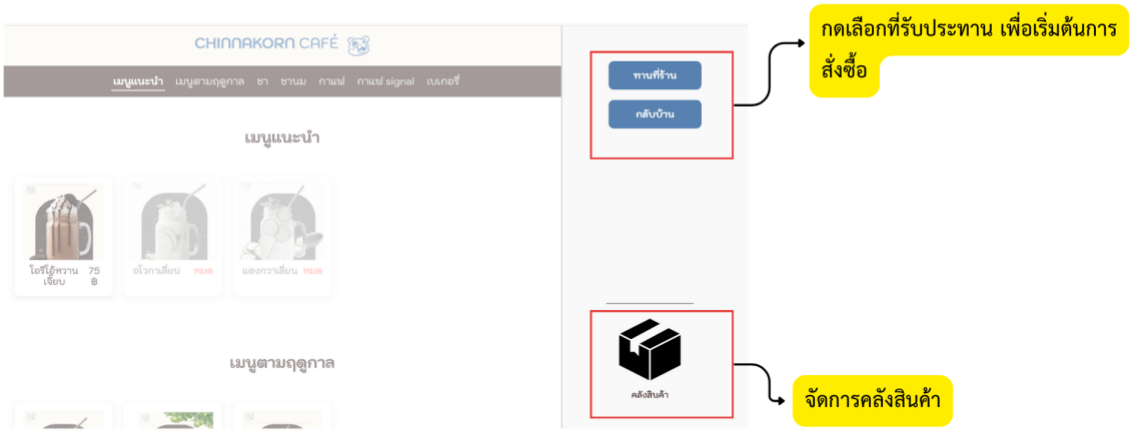


หลังจากผู้ใช้ทำการชำระเงินเรียบร้อยแล้ว ระบบจะนำผู้ใช้เข้าสู่ **หน้ายืนยันการชำระเงิน** เพื่อแสดงผลยืนยันว่าผู้ใช้ได้ทำการสั่งซื้อและชำระเงินสำเร็จ โดยหน้าดังกล่าวจะแสดงข้อความยืนยันการทำรายการ พร้อมราคาคำสั่งซื้อ และคำขอบคุณที่มาใช้บริการ เพื่อให้ผู้ใช้มั่นใจว่าการชำระเงินได้ดำเนินการเสร็จสมบูรณ์

เมื่อแสดงหน้าดังกล่าวครบถ้วน ระบบจะทำการ **นับถอยหลังประมาณ 10 วินาที** จากนั้นจะ กลับไปยังหน้าเริ่มต้นโดยอัตโนมัติ เพื่อเตรียมพร้อมสำหรับการสั่งซื้อครั้งถัดไป ทั้งนี้ หากผู้ใช้ต้องการทำการสั่งซื้อสินค้าใหม่ทันที สามารถกดปุ่ม **“สั่งซื้ออีกครั้ง”** เพื่อให้ระบบนำกลับไปยังหน้าเริ่มต้นโดยไม่ต้องรอเวลาได้เช่นกัน

พนักงานแคชเชียร์

หน้าเริ่มต้นของพนักงานแคชเชียร์



เมื่อเข้าสู่ระบบ จะปรากฏหน้าหลักซึ่งมี การใช้งานหลัก 2 ส่วน ได้แก่

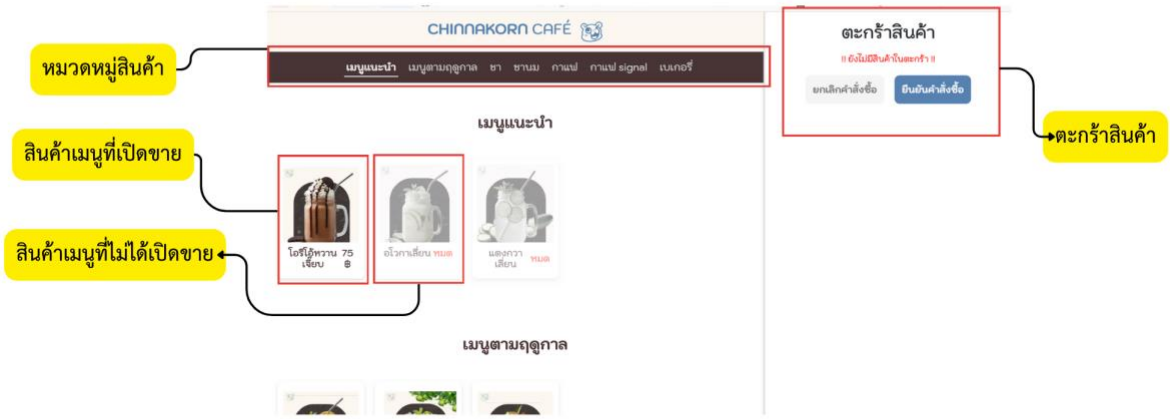
1. **โหมดการสั่งสินค้า**

   สำหรับทำการสั่งเครื่องดื่มหรือเมนูอาหาร โดยสามารถเริ่มต้นได้จากการเลือก **สถานที่รับประทานอาหาร** (เช่น ทานที่นี่ หรือสั่งกลับบ้าน) เพื่อเข้าสู่ขั้นตอนการเลือกและสั่งซื้อสินค้า

2. **โหมดการจัดการคลังสินค้า**

   สำหรับจัดการวัตถุดิบหรือส่วนประกอบต่าง ๆ ของสินค้า โดยสามารถเข้ามาดำเนินการเพิ่ม แก้ไข หรือตรวจสอบปริมาณวัตถุดิบคงเหลือในระบบ เพื่อให้การจัดการสต็อกสินค้ามีความถูกต้องและทันสมัยอยู่เสมอ

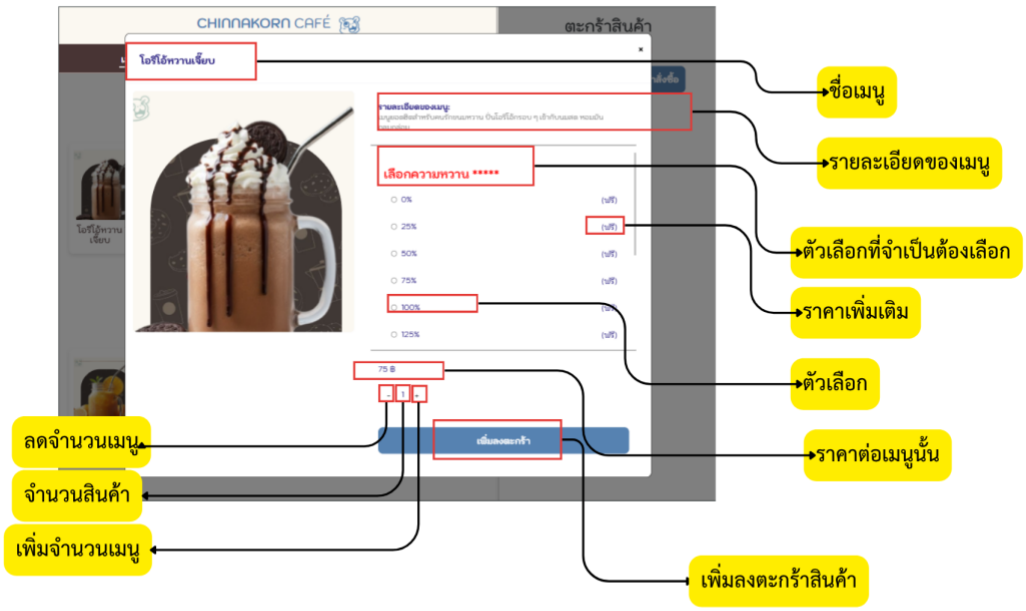
หน้าหลักของพนักงานแคชเชียร์ (main_for_cashier)



เมื่อผู้ใช้ทำการเลือกสถานที่สำหรับรับประทานอาหารเรียบร้อยแล้ว ระบบจะนำผู้ใช้เข้าสู่ **หน้าหลักสำหรับการเลือกรายการสินค้า** โดยในหน้านี้ผู้ใช้สามารถเลือกเมนูที่ต้องการสั่งซื้อได้อย่างสะดวก ผ่านการเลื่อนดูเมนูทั้งหมด หรือเลือกจาก **หมวดหมู่เมนู** ที่แสดงอยู่บริเวณด้านบนของหน้า เพื่อช่วยให้ค้นหาเมนูที่ต้องการได้รวดเร็วมากยิ่งขึ้น

ในหน้าดังกล่าว ระบบจะแสดงรายการสินค้าพร้อมชื่อเมนู ภาพประกอบ และราคาสินค้าอย่างชัดเจน เพื่อให้ผู้ใช้สามารถตัดสินใจเลือกซื้อได้ง่ายขึ้น ทั้งนี้ หากเมนูใด **หมดสต็อก** หรืออยู่ในสถานะ **ปิดการขาย** ระบบจะแสดงข้อความว่า “หมด” บนเมนูนั้น และจะ **ไม่อนุญาตให้ผู้ใช้ทำการสั่งซื้อ** เมนูดังกล่าว เพื่อป้องกันความผิดพลาดในการสั่งสินค้า

หน้าเลือกเมนูและปรับแต่ง



เมื่อผู้ใช้ทำการเลือกรายการสินค้าที่ต้องการสั่งซื้อเรียบร้อยแล้ว ระบบจะนำผู้ใช้เข้าสู่หน้าสำหรับเลือกตัวเลือกเพิ่มเติมของสินค้า (เช่น เครื่องดื่ม) ซึ่งหน้าดังกล่าวจะประกอบไปด้วยส่วนแสดงข้อมูลและองค์ประกอบที่สำคัญ ดังนี้

1. **ชื่อเมนูและรูปภาพสินค้า**

   แสดงชื่อเมนูพร้อมภาพประกอบ เพื่อให้ผู้ใช้สามารถระบุและเข้าใจลักษณะของสินค้าที่เลือกได้อย่างชัดเจน

2. **รายละเอียดสินค้า**

   แสดงข้อมูลเพิ่มเติมเกี่ยวกับสินค้า เช่น ส่วนผสม รสชาติ หรือคุณลักษณะเฉพาะของเมนูนั้น

3. **ตัวเลือกของสินค้า**

   ผู้ใช้สามารถเลือกตัวเลือกเพิ่มเติมที่เกี่ยวข้องกับสินค้า เช่น ระดับความหวาน ชนิดของนม หรือท็อปปิ้งเพิ่มเติม โดยมีรายละเอียดดังนี้

- หากเป็นตัวเลือกที่จำเป็นต้องเลือก ระบบจะแสดง **ตัวอักษรสีแดง** และ **เครื่องหมายดอกจัน (*)** เพื่อระบุความจำเป็นในการเลือก

- ในกรณีที่ตัวเลือกใด **หมดสต็อก** ระบบจะแสดงสถานะปิดและไม่อนุญาตให้ผู้ใช้ทำการเลือกได้

- สำหรับตัวเลือกที่มี **ค่าใช้จ่ายเพิ่มเติม** ระบบจะแสดงข้อความกำกับไว้ด้านหลังตัวเลือก เพื่อแจ้งจำนวนเงินที่ต้องชำระเพิ่มอย่างชัดเจน

4. **ส่วนการจัดการคำสั่งซื้อ**

   แสดงจำนวนสินค้า ราคา ปุ่มเพิ่มหรือลดจำนวนสินค้า และปุ่มสำหรับเพิ่มสินค้าลงในตะกร้า
   เพื่อให้ผู้ใช้สามารถปรับแต่งรายการสั่งซื้อได้อย่างสะดวกและถูกต้องก่อนยืนยันการสั่งซื้อ

หน้าตะกร้าสินค้า



เมื่อผู้ใช้กดเพิ่มรายการสินค้าที่ต้องการ ระบบจะทำการบันทึกรายการสินค้านั้นเข้าสู่ **ตะกร้าสินค้า**
โดยผู้ใช้สามารถเข้าสู่หน้าตะกร้าเพื่อดูรายละเอียดของสินค้าที่ได้เลือกไว้ได้ ซึ่งภายในตะกร้าจะแสดงข้อมูลสำคัญดังนี้

1. **ชื่อเมนูและตัวเลือกสินค้า**
   แสดงชื่อเมนูพร้อมรายละเอียดของตัวเลือกที่ผู้ใช้ได้ทำการเลือกไว้สำหรับสินค้านั้น
   เพื่อให้ผู้ใช้สามารถตรวจสอบความถูกต้องของรายการก่อนทำการสั่งซื้อ

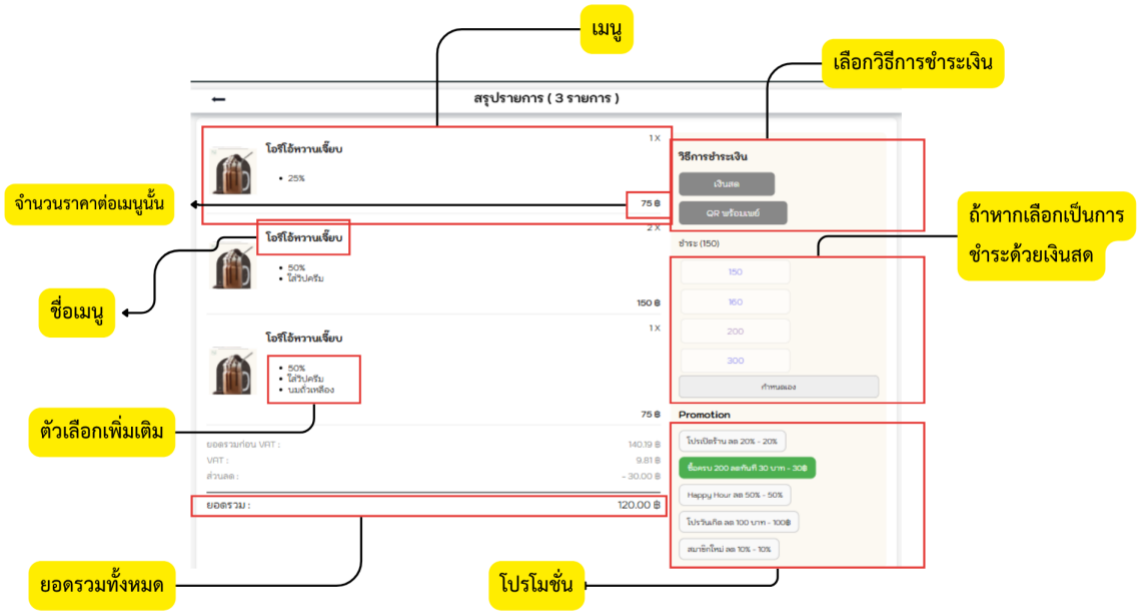
2. **ราคาสินค้า**
   แสดงราคาต่อหนึ่งหน่วยของสินค้า รวมถึงราคารวมของสินค้าทั้งหมดในตะกร้า เพื่อให้ผู้ใช้สามารถทราบยอดชำระโดยรวมได้อย่างชัดเจน

3. **การปรับจำนวนสินค้า**
   ผู้ใช้สามารถเพิ่มหรือลดจำนวนสินค้าภายในตะกร้าได้ตามต้องการ หากผู้ใช้ปรับจำนวนสินค้าลงเป็นศูนย์ (0)
   ระบบจะทำการลบสินค้านั้นออกจากตะกร้าโดยอัตโนมัติ

4. **การยืนยันและยกเลิกคำสั่งซื้อ**

- หากผู้ใช้ต้องการดำเนินการสั่งซื้อ ระบบมีปุ่ม **“ยืนยันคำสั่งซื้อ”** เพื่อเข้าสู่ขั้นตอนถัดไปของการสั่งซื้อสินค้า
  ในกรณีที่ผู้ใช้ไม่ประสงค์จะสั่งซื้อ ระบบมีปุ่ม **“ยกเลิกคำสั่งซื้อ”** เพื่อยกเลิกรายการทั้งหมด
  และระบบจะนำผู้ใช้กลับไปยังหน้าแรกของระบบโดยอัตโนมัติ

หน้ายืนยันคำสั่งซื้อ



หลังจากผู้ใช้กดยืนยันการสั่งซื้อ ระบบจะนำผู้ใช้เข้าสู่ **หน้าชำระเงิน**
ซึ่งในหน้านี้จะแสดงรายละเอียดของรายการสินค้าที่ผู้ใช้ได้ทำการเลือกไว้ทั้งหมด ประกอบด้วย **ชื่อเมนู ตัวเลือกของเมนู จำนวนสินค้าที่สั่งซื้อ ราคาต่อหน่วย** และ **ราคารวมของแต่ละรายการ** เพื่อให้ผู้ใช้สามารถตรวจสอบความถูกต้องของคำสั่งซื้อได้ก่อนทำการชำระเงิน

นอกจากนี้ ระบบจะแสดง **สรุปรายละเอียดราคาทั้งหมด** ได้แก่

- **ราคารวมก่อนภาษีมูลค่าเพิ่ม (ก่อน VAT)**
- **จำนวนภาษีมูลค่าเพิ่ม (VAT)**
- **ราคารวมสุทธิ (ยอดชำระทั้งหมด)**

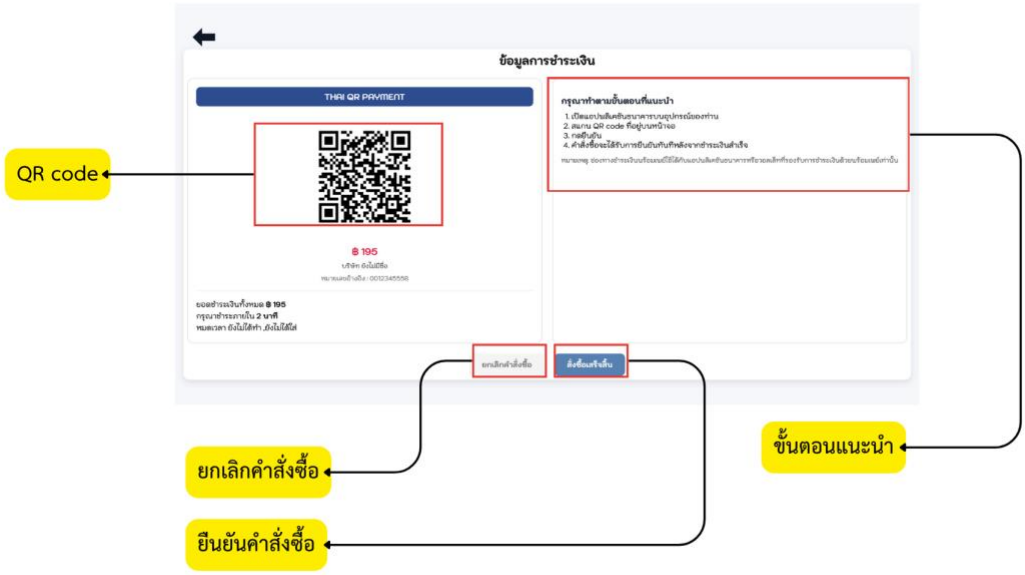
ภายในหน้านี้ ผู้ใช้สามารถเลือกวิธีการชำระเงินได้ โดยปัจจุบันระบบรองรับ **2 รูปแบบหลักของการชำระเงิน** ได้แก่

1. **การชำระเงินผ่าน QR Code Payment** – ระบบจะแสดงรหัส QR
   สำหรับให้ผู้ใช้สแกนและดำเนินการชำระเงินผ่านแอปพลิเคชันธนาคารได้โดยตรง เพื่อความสะดวกและรวดเร็ว

2. **การชำระเงินด้วยเงินสด (Cash Payment)** – ระบบจะมีปุ่มให้ผู้ใช้ระบุจำนวนเงินที่ลูกค้าชำระเข้ามา โดยแบ่งเป็น **5 ตัวเลือกหลัก** ได้แก่
   - ชำระเงิน **พอดีกับยอดรวมทั้งหมด**
   - ชำระเงิน **มากกว่ายอดรวมทั้งหมด 3 ระดับ** (เช่น +10 บาท, +20 บาท, +50 บาท เป็นต้น)

- หากลูกค้าชำระเงินเป็นจำนวนอื่นที่ไม่อยู่ในตัวเลือกดังกล่าว ระบบจะมีช่องให้ **กรอกจำนวนเงินที่ชำระจริง** เพื่อความยืดหยุ่นในการทำรายการ

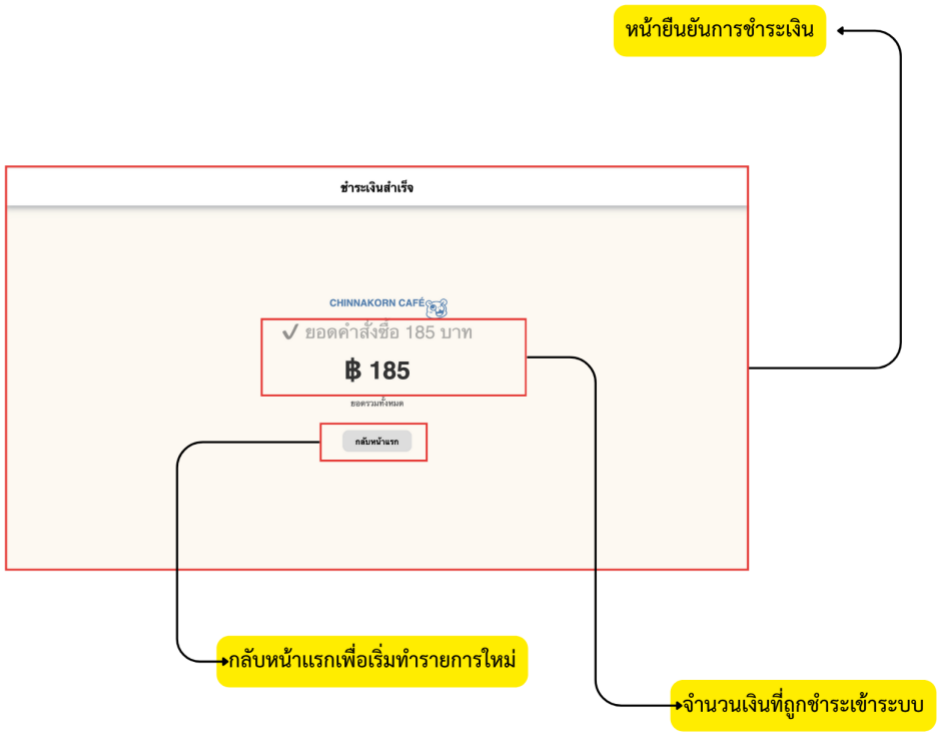
หน้าชำระเงิน



หากมีการเลือกวิธีการชำระเงินด้วย QR code ในหน้าชำระเงิน ระบบจะแสดง **QR Code สำหรับการชำระเงิน** พร้อมทั้ง **รายละเอียดราคาสินค้า** เพื่อให้ผู้ใช้สามารถตรวจสอบความถูกต้องของยอดชำระก่อนดำเนินการชำระเงินจริง นอกจากนี้ ภายในหน้ายังมีการแสดง **ขั้นตอนและวิธีการชำระเงิน** อย่างชัดเจน เพื่อให้ผู้ใช้สามารถทำตามได้อย่างถูกต้องและสะดวกต่อการใช้งาน

เมื่อผู้ใช้ทำการชำระเงินเรียบร้อยแล้ว สามารถกดปุ่ม **“สั่งซื้อเสร็จสิ้น”** เพื่อยืนยันการทำรายการและเข้าสู่ขั้นตอนถัดไปของระบบได้ทันที ในกรณีที่เกิดปัญหาระหว่างการทำรายการ หรือไม่

หน้ายืนยันการชำระเงิน หรือ ชำระเงินสำเร็จ



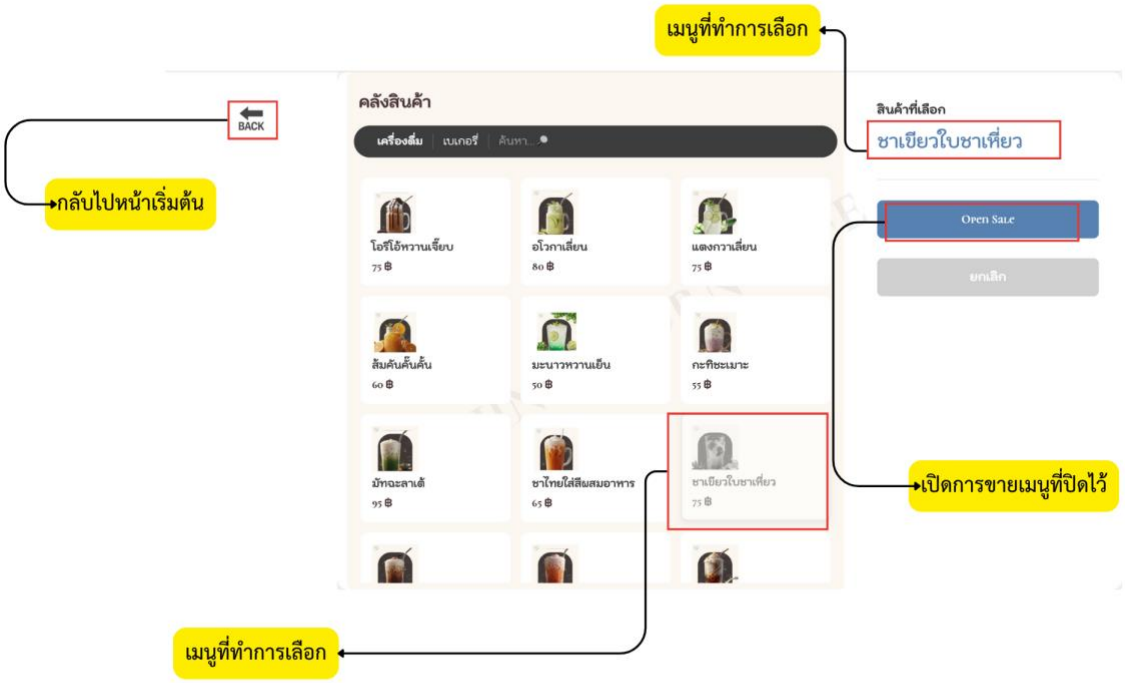
หลังจากผู้ใช้ทำการชำระเงินเรียบร้อยแล้ว ระบบจะนำผู้ใช้เข้าสู่ **หน้ายืนยันการชำระเงิน** เพื่อแสดงผลยืนยันว่าผู้ใช้ได้ทำการสั่งซื้อและชำระเงินสำเร็จ โดยหน้าดังกล่าวจะแสดงข้อความยืนยันการทำรายการ พร้อมราคาคำสั่งซื้อ

ในกรณีที่ผู้ใช้เลือกการชำระเงินด้วยเงินสด และมีจำนวนเงินทอน ที่ต้องคืนให้แก่ลูกค้า ระบบจะแสดง ข้อความแจ้งเตือนด้วยตัวอักษรสีแดง พร้อมแสดง จำนวนเงินทอนที่ต้องคืนให้ลูกค้า อย่างชัดเจน เพื่อช่วยลดความผิดพลาดในการให้บริการและเพิ่มความถูกต้องในขั้นตอนการทำรายการชำระเงิน

เมื่อแสดงหน้าดังกล่าวครบถ้วน ระบบจะทำการ นับถอยหลังประมาณ 10 วินาที จากนั้นจะ กลับไปยังหน้าเริ่มต้นโดยอัตโนมัติ เพื่อเตรียมพร้อมสำหรับการสั่งซื้อครั้งถัดไป ทั้งนี้ หากผู้ใช้ต้องการทำการสั่งซื้อสินค้าใหม่ทันที สามารถกดปุ่ม **“สั่งซื้ออีกครั้ง”** เพื่อให้ระบบนำกลับไปยังหน้าเริ่มต้นโดยไม่ต้องรอเวลาได้เช่นกัน

หน้าจัดการคลังสินค้าสำหรับพนักงานแคชเชียร์


สินค้าที่เลือก
ตัวเลือกหมวดหมู่
ค้นหาชื่อสินค้า
BACK
คลังสินค้า
เครื่องดื่ม
เบเกอรี่
สินค้าที่เลือก
เลือกสินค้า
ยกเลิก
กลับไปหน้าเริ่มต้น


เมนูที่ทำการเลือก
BACK
คลังสินค้า
เครื่องดื่ม
เบเกอรี่
สินค้าที่เลือก
ชาเขียวใบชาเหี่ยว
Open Sale
ยกเลิก
โอรีโอ้หวานเจี๊ยบ
75 ฿
อโวกาเลี่ยน
80 ฿
แตงกวาเลี่ยน
75 ฿
ส้มคันคั้นคั้น
60 ฿
มะนาวหวานเย็น
50 ฿
กะทิชะเมาะ
55 ฿
มัทฉะลาเต้
95 ฿
ชาไทยใส่สีผสมอาหาร
65 ฿
ชาเขียวใบชาเหี่ยว
75 ฿
เปิดการขายเมนูที่ปิดไว้
กลับไปหน้าเริ่มต้น
เมนูที่ทำการเลือก


เมนูที่ทำการเลือก
คลังสินค้า
BACK
เครื่องดื่ม
เบเกอรี่
ค้นหา...
สินค้าที่เลือก
มะนาวหวานเย็น
Close Sale
ยกเลิก
กลับไปหน้าเริ่มต้น
เมนูที่ทำการเลือก
ปิดการขายเมนูที่เปิดไว้


เมนูที่ทำการเลือก
คลังสินค้า
BACK
เครื่องดื่ม
เบเกอรี่
ค้นหา...
สินค้าที่เลือก
คุกทายเสี่ยงกี้
Stock: 120
Edit
ยกเลิก
กลับไปหน้าเริ่มต้น
เมนูที่ทำการเลือก
ลดจำนวน stcok
เพิ่มจำนวน stcok
กดเปลี่ยนจำนวน stock
จำนวน stock

ในกรณีที่ผู้ใช้เลือก **โหมดการจัดการคลังสินค้า (Inventory Management)** ระบบจะแสดง **หน้าคลังสินค้า** ซึ่งแบ่งหมวดหมู่คลังสินค้าออกเป็น **2 ส่วนหลัก** ได้แก่

1. **เครื่องดื่ม (Beverages)**

2. **เบเกอรี่ (Bakery)**

สำหรับหมวดเมนู **เครื่องดื่ม** ผู้ใช้สามารถทำการ **เปิดหรือปิดการขายของเมนูนั้นได้โดยตรง** โดยระบบจะแสดงสถานะปัจจุบันของเมนู ดังนี้

- หากเลือกเมนูที่อยู่ในสถานะ **ปิดการขาย** ระบบจะแสดงปุ่มให้ผู้ใช้สามารถ **เปิดการขาย** ของเมนูนั้นได้

- หากเลือกเมนูที่อยู่ในสถานะ **เปิดการขาย** ระบบจะแสดงปุ่มให้ผู้ใช้สามารถ **ปิดการขาย** ของเมนูนั้นได้เช่นกัน

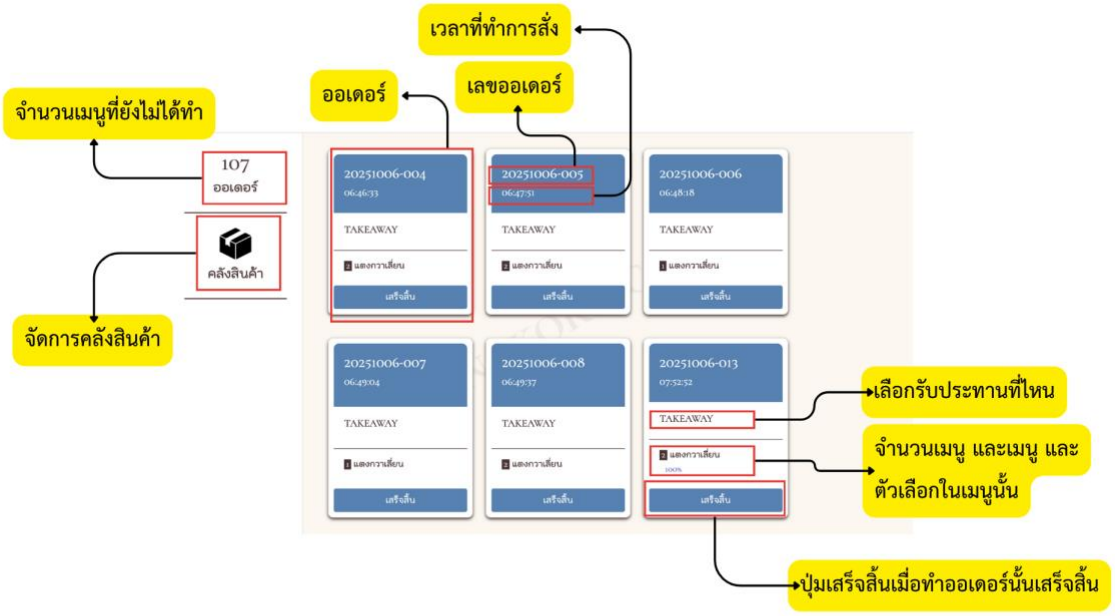
สำหรับหมวดเมนู **เบเกอรี่** ผู้ใช้สามารถทำการ **เพิ่มจำนวนสินค้าคงเหลือในสต็อก** เพื่อเตรียมพร้อมสำหรับการขายได้โดยตรง โดยระบบจะแสดง **สถานะปัจจุบันของแต่ละเมนู** ดังนี้

- หากเมนูอยู่ในสถานะ **ปิดการขาย** หมายความว่า **สินค้าคงเหลือในสต็อกหมดแล้ว**
  และไม่สามารถทำการขายได้จนกว่าจะเพิ่มจำนวนสินค้าลงในสต็อก

และมีปุ่มกลับไปหน้าเริ่มต้นสำหรับเลือกสั่งเมนูหรือจัดการคลังสินค้า

พนักงานบาริสต้า

หน้าหลักของพนักงานบาริสต้า



ระบบจะแสดง **รายการสินค้า หรือคำสั่งซื้อ** (Orders) ที่ลูกค้าได้ทำการสั่งเข้ามา โดยรายการออเดอร์จะถูก **จัดเรียงตามลำดับเวลาและหมายเลขออเดอร์** ซึ่งออเดอร์ที่เข้ามาก่อนจะอยู่ลำดับแรก และเรียงตามลำดับเวลาถัดไปตามลำดับการเข้ามาของคำสั่งซื้อ

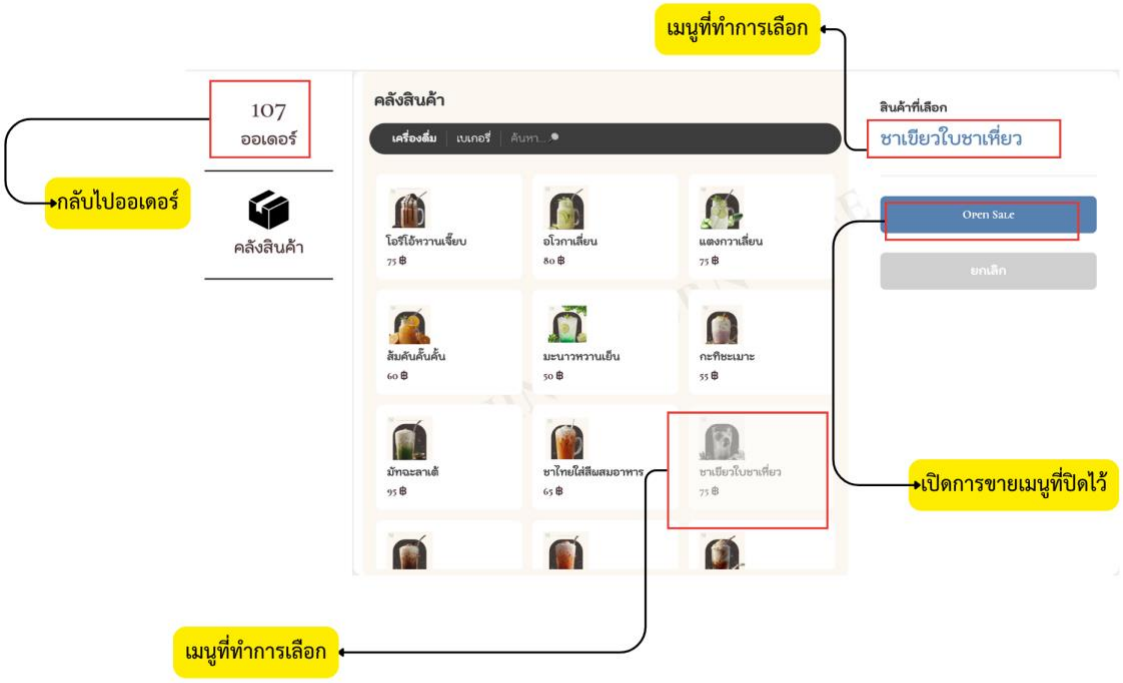
ในแต่ละรายการออเดอร์จะแสดงข้อมูลสำคัญดังนี้

- **สถานที่รับประทานอาหาร** ที่ลูกค้าได้เลือก
- **รายการเครื่องดื่มและเมนูที่ลูกค้าสั่ง**

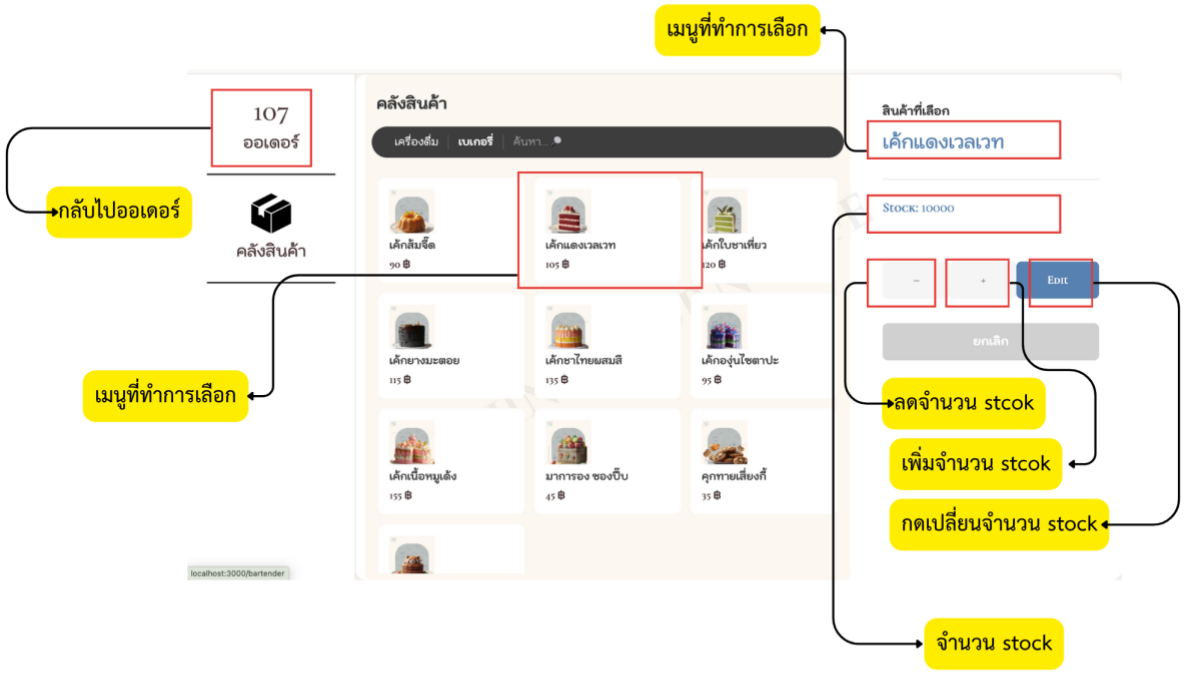
เมื่อพนักงานบาริสต้าทำการจัดเตรียมและเสร็จสิ้นออเดอร์แล้ว สามารถกดปุ่ม **“เสร็จสิ้น”** เพื่อทำการปิดออเดอร์ดังกล่าว ระบบจะบันทึกสถานะออเดอร์เป็น **เสร็จสิ้น** และออเดอร์นั้นจะไม่ปรากฏในรายการที่ต้องทำอีกต่อไป

หน้าจัดการคลังสินค้าของบาริสต้า









ในกรณีที่ผู้ใช้เลือก **โหมดการจัดการคลังสินค้า (Inventory Management)** ระบบจะแสดง **หน้าคลังสินค้า** ซึ่งแบ่งหมวดหมู่คลังสินค้าออกเป็น **2 ส่วนหลัก** ได้แก่

3. **เครื่องดื่ม (Beverages)**

4. **เบเกอรี่ (Bakery)**

สำหรับหมวดเมนู **เครื่องดื่ม** ผู้ใช้สามารถทำการ **เปิดหรือปิดการขายของเมนูนั้นได้โดยตรง** โดยระบบจะแสดงสถานะปัจจุบันของเมนู ดังนี้

- หากเลือกเมนูที่อยู่ในสถานะ **ปิดการขาย** ระบบจะแสดงปุ่มให้ผู้ใช้สามารถ **เปิดการขาย** ของเมนูนั้นได้
- หากเลือกเมนูที่อยู่ในสถานะ **เปิดการขาย** ระบบจะแสดงปุ่มให้ผู้ใช้สามารถ **ปิดการขาย** ของเมนูนั้นได้เช่นกัน

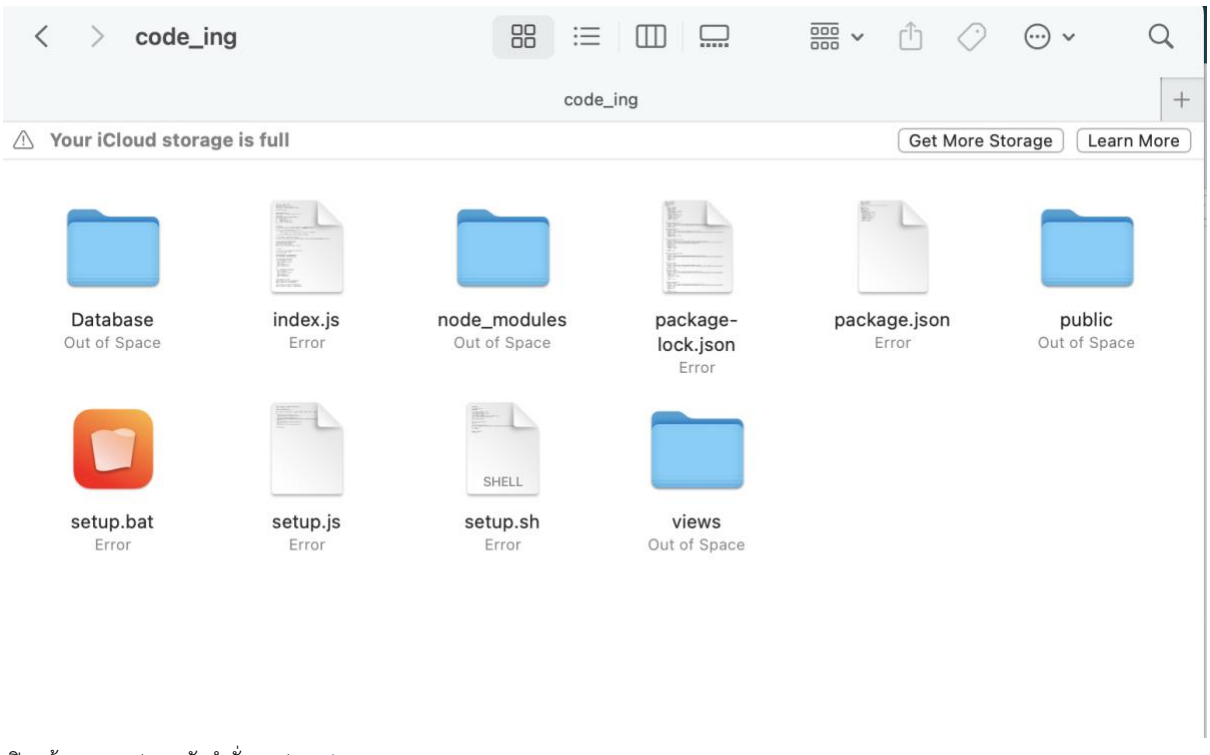
สำหรับหมวดเมนู **เบเกอรี่** ผู้ใช้สามารถทำการ **เพิ่มจำนวนสินค้าคงเหลือในสต็อก** เพื่อเตรียมพร้อมสำหรับการขายได้โดยตรง โดยระบบจะแสดง **สถานะปัจจุบันของแต่ละเมนู** ดังนี้

- หากเมนูอยู่ในสถานะ **ปิดการขาย** หมายความว่า **สินค้าคงเหลือในสต็อกหมดแล้ว**
  และไม่สามารถทำการขายได้จนกว่าจะเพิ่มจำนวนสินค้าลงในสต็อก

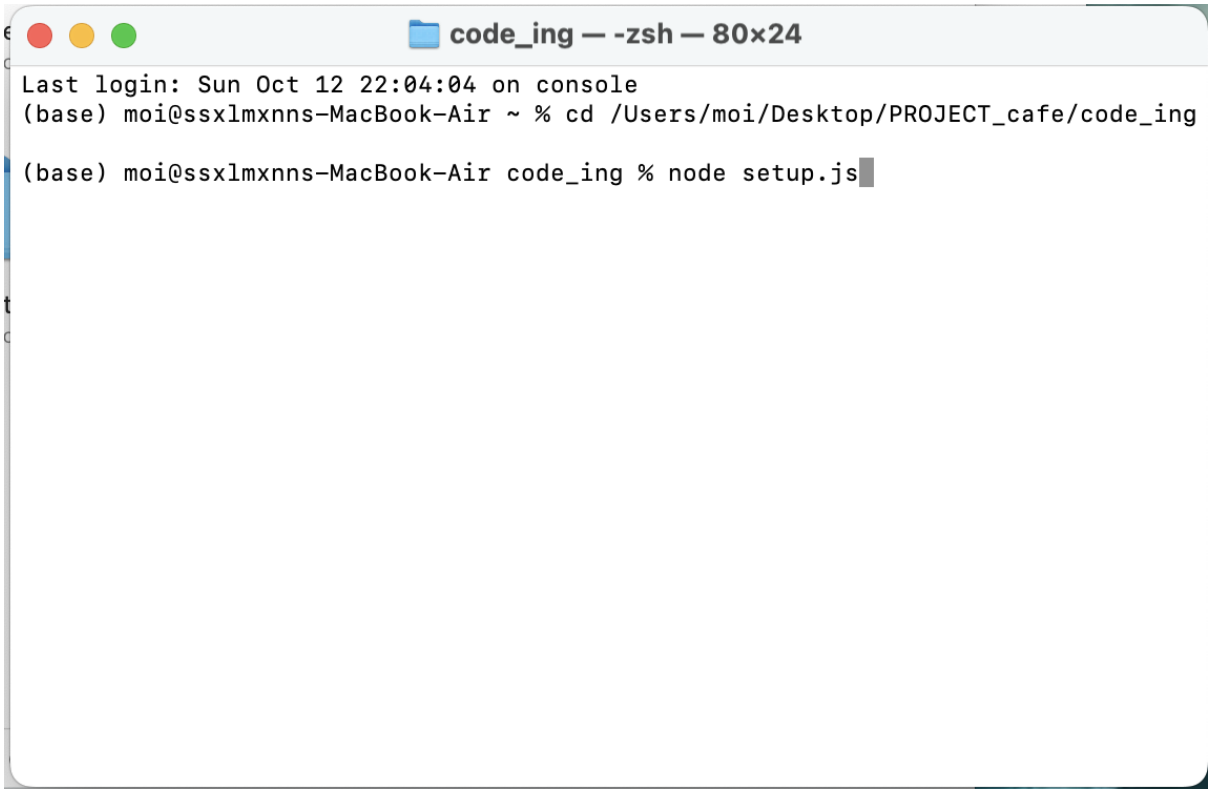
คู่มือการใช้งาน การติดตั้งระบบ

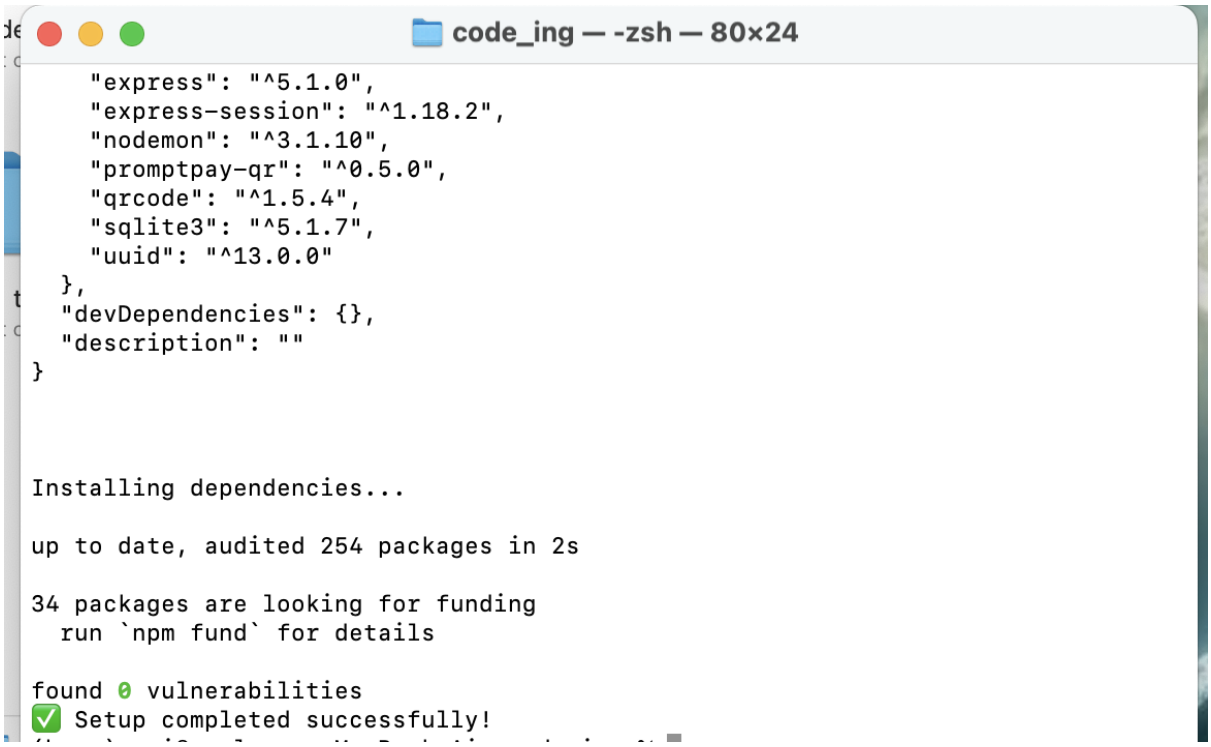
วิธีการติดตั้ง

1. ทำการดาวโหลดไฟล์



2. เปิดหน้า terminal และรันคำสั่ง node index.js



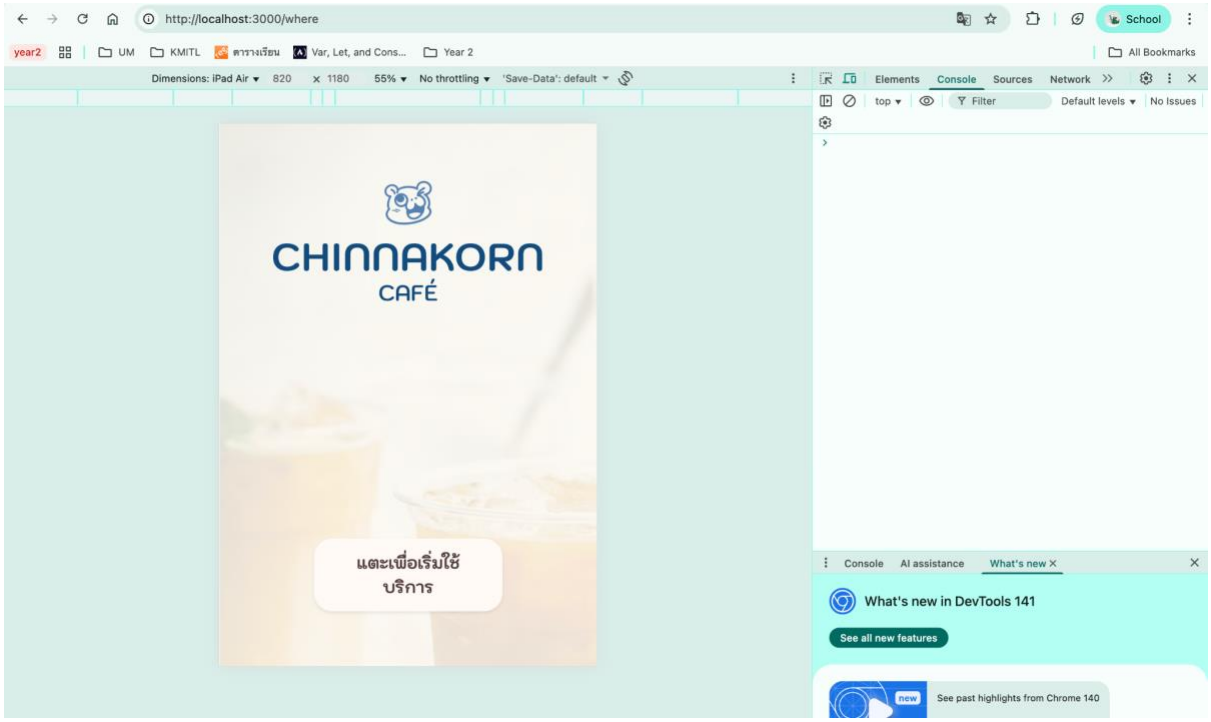


3. เปิดหน้า browser ที่ต้องการ (http://localhost:3000)

http://localhost:3000/ ⟶ สำหรับลูกค้า

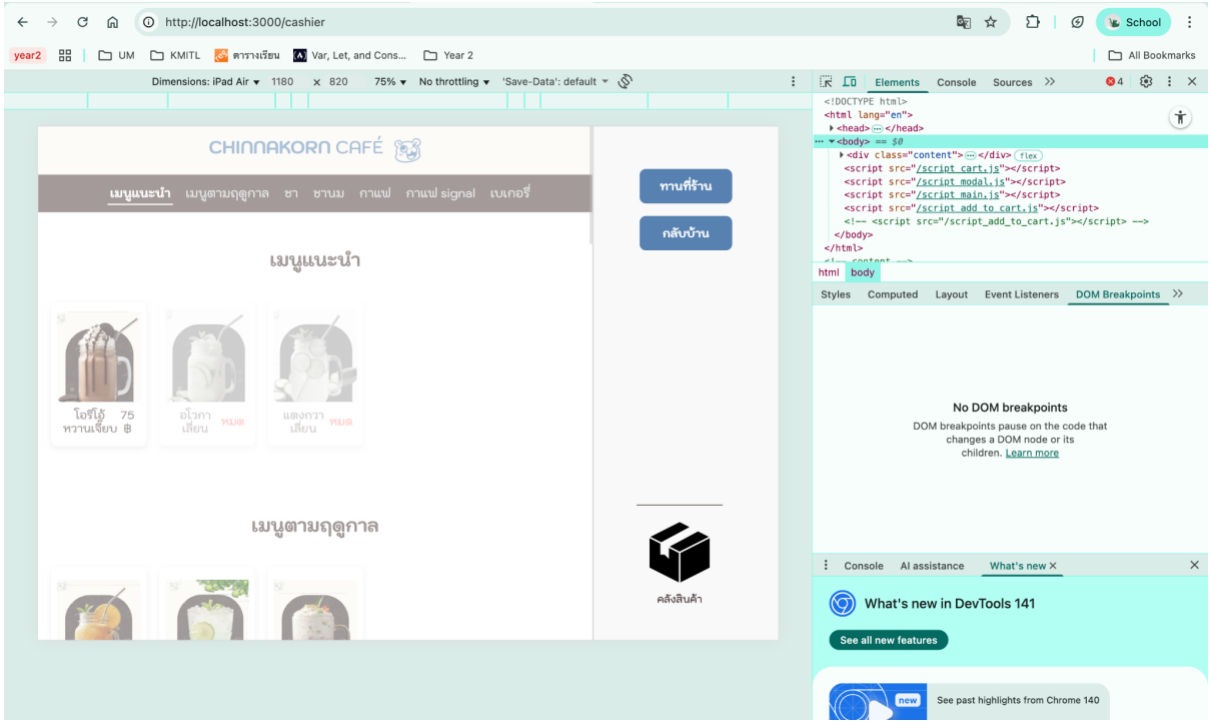
http://localhost:3000/cashier ⟶ สำหรับพนักงานแคชเชียร์ http://localhost:3000/bartender ⟶ สำหรับพนักงานบาริสต้า

หน้าสำหรับลูกค้า



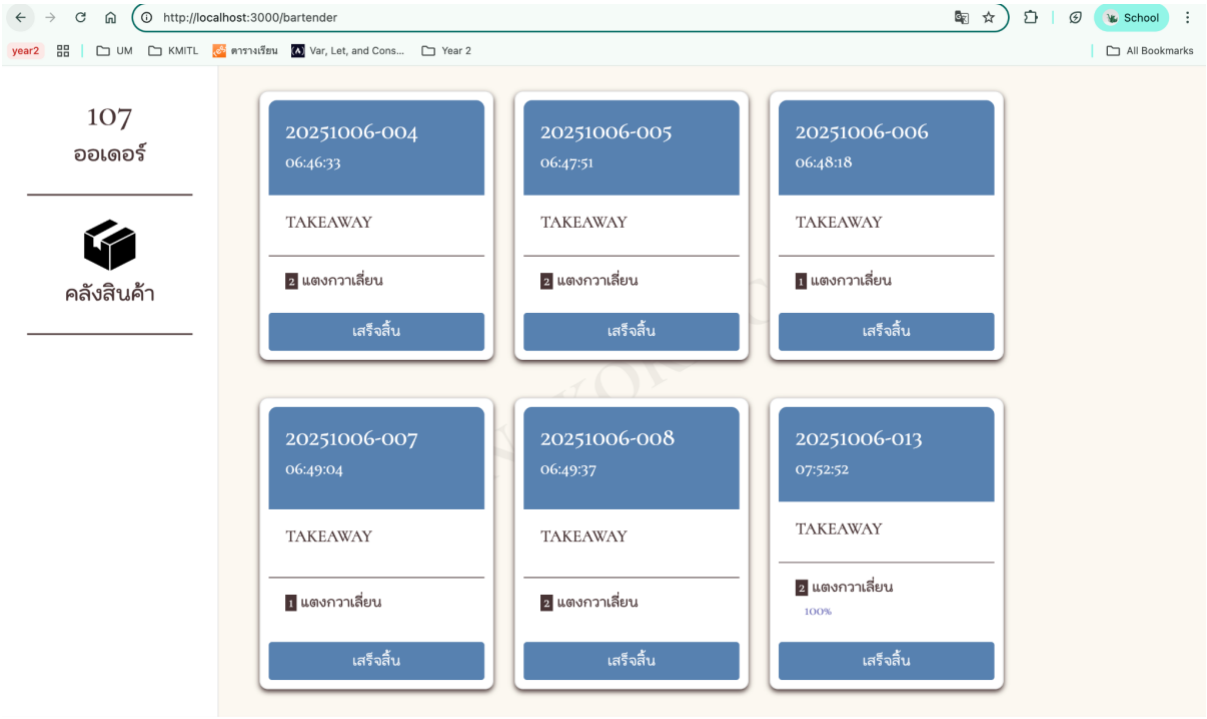
** ภาพที่เห็นข้างต้นคือการปรับขนาดให้เหมาะสำหรับอุปกรณ์ที่จะทำการใช้งานจริง **

หน้าสำหรับพนักงานแคชเชียร์



** ภาพที่เห็นข้างต้นคือการปรับขนาดให้เหมาะสำหรับอุปกรณ์ที่จะทำการใช้งานจริง **

หน้าสำหรับพนักงาน บาริสต้า



ปัญหาและแนวทางการปรับปรุง

1. **การแสดงผลตะกร้าสินค้า :**

ปัจจุบันระบบตะกร้าสินค้าต้องทำการรีเฟรชหน้าเว็บก่อนจึงจะแสดงรายการที่เพิ่มเข้ามา

แนวทางปรับปรุง:

ในอนาคตสามารถปรับให้หน้าเว็บอัปเดตรายการตะกร้าแบบ เรียลไทม์ (Real-time update) โดยใช้ AJAX หรือ Fetch API เพื่อให้ประสบการณ์ผู้ใช้ราบรื่นมากขึ้นโดยไม่ต้องรีเฟรชหน้า

2. **การจัดการสต็อกวัตถุดิบ :**

ขณะนี้ระบบสามารถลดจำนวนวัตถุดิบได้อัตโนมัติเมื่อมีการสั่งเมนู แต่ยังไม่มีระบบเพิ่มสต็อกผ่านหน้าแอปพลิเคชัน

แนวทางปรับปรุง:

วางแผนเพิ่มฟังก์ชันให้บาริสต้าหรือผู้จัดการสามารถ เพิ่ม/แก้ไขสต็อกวัตถุดิบผ่านหน้าเว็บ ได้โดยตรง โดยจะช่วยลดความจำเป็นในการเข้าไปแก้ไขผ่านฐานข้อมูลโดยตรง และทำให้การจัดการร้านสมบูรณ์ยิ่งขึ้น